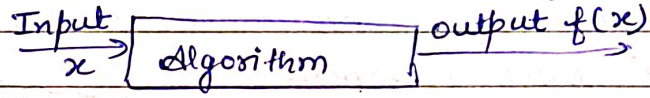
UNIT-1

Basic Concepts and Automata Theory :-

Introduction to Theory of computation – Aut... computability and complexity, Alphabet, Symbol, st... mal languages, Deterministic Finite Automaton (DFA) – D..., Representation, Acceptability of String & Language ... Deterministic Finite automata (NFA), Equivalence of DFA ... FA with ε-transition, Equivalence of NFA's with and ... ε-transition, Finite Automata with output – Moore M... ealy Machine, Equivalence of Moore & Mealy Machi... inimization of Finite Automata, Myhill Nerode Theorem.

Introduction to Theory of Computation :-

- Computation - It is a step by step solution to a problem. It is carried out by executing programs.
- Theory of computation is theory about programs.
- Program is nothing but expression of algorithms in a particular language i.e. Programs express algorithms.
- An algorithm is a recipe for carrying out input to output transformation
- Function : A mapping from domain (input) to range (output)



- An algorithm computes a function.
- Example of function :

is-prime : number → {yes, no}

$$\text{is-prime}(n) := \begin{cases} \text{yes}, & \text{if } n \text{ is a prime} \\ \text{no} & \text{if } n \text{ is not a prime} \end{cases}$$

function definition

- Function definition does not say anything about how to come up with an answer.
- So we need algorithm which will tell us how to come up with an answer. The steps of algorithm should be simple operations which will be carried out on a computer.

Basic Goal : To identify the class of functions which do not admit any algorithms to compute them.

- At the end of this course we will have the Knowledge of the techniques to demonstrate that a function does not admit any algorithms to compute.

1- Set Membership problem : Given a set S and an input a. Decide whether $a \in S$ or not.

- Let us define a set which is associated to any function f as follows:-

$$graph(f) = \{(a,b) \mid f(a)=b\}$$

- Suppose we show that there is no algorithm to solve the set membership problem graph(f) then we conclude that there is no algorithm to compute f.

Because if there is an algorithm to compute f then for input (a,b) we can compute f(a) using the algorithm & suppose $b=f(a) \iff (a,b) \in graph(f)$

THE ENTIRE COURSE CONCENTRATE ON SET & THEIR MEMBESHIP FUNCTIONS RATHER THAN ONLY FUNCTIONS.

- The sets are going to be the sets of finite strings.

COMPONENTS OF THEORY OF COMPUTATION :-

1. COMPLIEXITY THEORY :-

- The main question asked in this area : what makes some problems computationally hard & other problems easy?
- We have to classify problems according to their degree of difficulty. We need to give a rigorous proof that problems that seem to be "hard" are really "hard".

2. COMPUTABILITY THEORY :-

- In 1930 Godel Turing & Church discovered that some of the mathematical problems (fundamental problems) can't be solved by computer.
- This theory helps us to classify the problems as being solvable or unsolvable.

3. AUTOMATA THEORY :-

- This theory deals with definitions & properties of different types of computational models like
  - Finite Automata : used in text processing, compiler etc.
  - Context-Free Grammars : used in programming language AI etc.
  - Turing Machine : Model of real computers.
- Central Question : Do all these models have same powers, or one model solve more problems than the others.

CENTRAL CONCEPTS OF AUTOMATA THEORY

1. Alphabet : An alphabet is a finite, non empty symbols.
   We use the symbol Σ for an alphabet
   - $\Sigma = \{0, 1\}$ binary alphabet.
   - $\Sigma = \{a, b, c, \ldots, z\}$ set of all lower case letters
   - The set of all ASCII characters.

2. Strings : A finite sequence of symbols chosen from some alphabet.
   Normally denoted by w, x, y, z.
   Eg: If $\Sigma = \{0, 1\}$ then 1011, 111, 11000 are strings
   If $\Sigma = \{a, b, c, \ldots, z\}$ then abc, abd, xyz are strings

Empty String : String with zero occurrences of symbols denoted by ϵ

Length of a string :- The number of positions for symbols in string

If $w = 1011$ then $|w| = 4$

Powers of an alphabet : If Σ : an alphabet

Σ^k : set of strings of length K whose symbol is in Σ.

$\Sigma^0 = \epsilon$

If $\Sigma = \{0, 1\}$ then $\Sigma^1 = \{0, 1\}$

$\Sigma^2 = \{00, 01, 10, 11\}$ $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

NOTE: Difference between Σ & Σ^1 ← set of strings with members as strings 0, 1 of length 1

alphabet set with members 0, 1

Σ^* : set of all strings over an alphabet Σ

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \ldots\ldots\ldots\ldots$

Σ^+ : set of non empty strings from alphabet Σ

$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \ldots\ldots\ldots$

$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$

$\emptyset \neq \{\epsilon\}$ — contains one string

contains no string

3. **CONCATENATION OF STRINGS :-**

Let x, y : strings

xy : concatenation of x & y

Eg. : If $x = 1101$, $y = 0011$ then $xy = 1101 0011$, $yx = 0011 1101$

For any string w, $\epsilon w = w\epsilon = w$

ϵ : identity for concatenation

3. **LANGUAGES :-** set of strings all of which are chosen from some Σ^*, where Σ is a particular alphabet.

If Σ is an alphabet & $L \subseteq \Sigma^*$, then L is a language over Σ

Examples: English language, C language

→ Language of all strings consisting of n 0's followed by n 1's, for some $n \geq 0$

$\{\epsilon, 01, 0011, 000111, \ldots\}$

→ Language of all strings of 0's and 1's with an equal no. of each.

$\{\epsilon, 01, 10, 0011, 0110, 1001, \ldots\}$

→ Language of binary nos whose value is a prime

$\{10, 11, 101, 1011, \ldots\}$

→ Σ^* : language for any alphabet Σ

→ $\emptyset$: Empty language over any alphabet.

→ $\{\epsilon\}$: language consisting of only empty strings is a language over any alphabet.

NOTE : $\emptyset \neq \{\epsilon\}$

$\emptyset$ contains no string

$\{\epsilon\}$ contains one string

4. Problem (set membership problem)

Given Σ: an alphabet
L: language over Σ
w: string in Σ^*

Decide whether or not w is in L

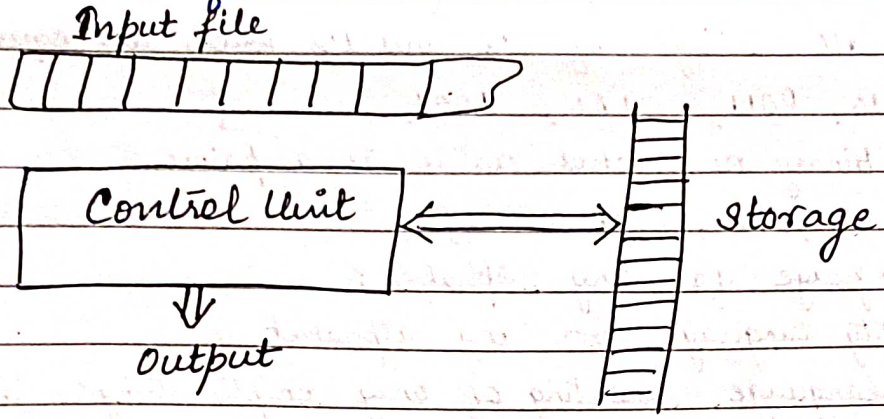
AUTOMATA :

→ An automaton is an abstract model of a digital computer. As such, every automaton includes some essential features.

→ It has a mechanism for reading input.

→ It will be assumed that the input is a string over a given alphabet written on an input file which automaton can read but not change.

→ The input mechanism can read the input file from left to right, one symbol at a time. It can also detect the end of input string.
→ The automaton can read & change the contents of storage cell.
→ Finally, the automaton has control unit which can be in any one of a finite no. of internal states.

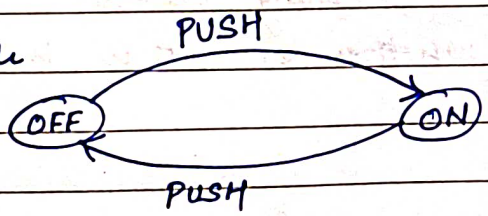


FINITE AUTOMATA (plural of automaton) something capable of acting automatically without an external motive.

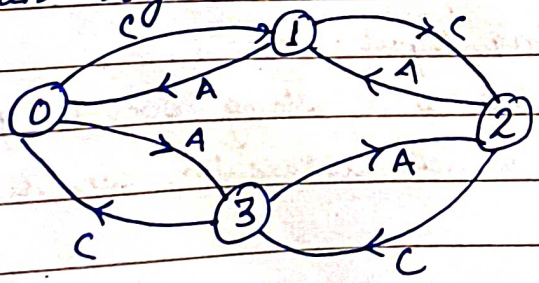
→ It is a computational device (model) which has finite amount of memory (others computational devices: calculator, cell phone
→ It has finite no. of states which does not depend on input size.
→ At a particular time it remembers only the current state.

Examples:

a) An electric switch



b) Fan regulator

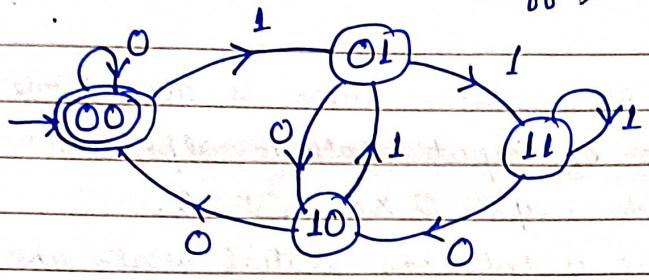


c) We are looking for a set of strings defined as

$L = \{ x : x \text{ is a binary string divisible by } 4 \}$

$= \{ 100, 1000, 1100, \ldots \}$

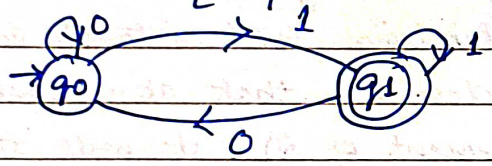
Note: All binary nos which are divisible by 4 have "00" at the end (00 as the suffix)



Let us store the info. of last two bits in the states.

d) Consider the following language

$L = \{ w / w \text{ ends with a } 1 \}$



Deterministic Finite Automata (DFA) :-

Designing a DFA is like writing a program. In a program, the current state of a program is given by :-

1) values stored in variables.
2) current line of execution.

In case of DFA, variables are not used. Everything is remembered through explicit states. Transition function of DFA is similar to algorithm. While designing DFA one has to concentrate on four issues.

1. No. of states
2. Transition functions
3. Start state
4. Final state.

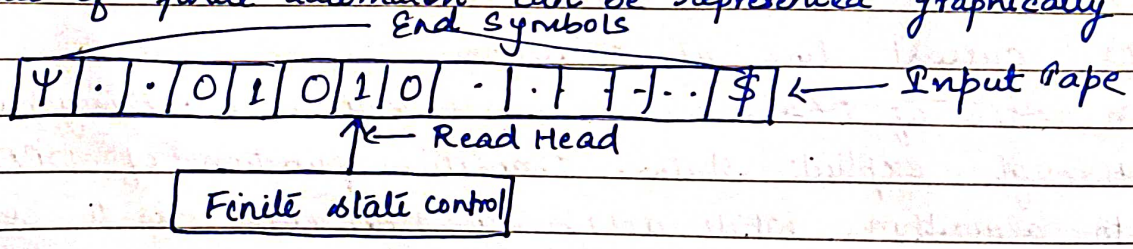
In a deterministic finite automaton there is only one next state 'q0' on input 'a'. The maximum no. outgoing transition from a particular state are no. of different symbols from Σ.

Definition A finite automaton is defined as 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where,

$Q \longrightarrow$ finite non empty set of states (there is atleast one state)
$\Sigma \rightarrow$ finite non empty set of input called alphabet
$\delta \rightarrow$ transition function which maps $Q \times \Sigma$ into Q.
$q_0 \rightarrow$ initial state (i.e. there is only one initial state and $q_0 \in Q$)
$F \rightarrow$ set of final states and $F \subseteq Q$

1. The transition function describes the change in state during the transitions. The transition function $Q \times \Sigma$ into Q can also be written as $Q \times \Sigma \rightarrow Q$ which describes that at a time a machine is in some state (element of Q), it will take some input (element of Σ), as a result it goes to some next state (element of Q).

2. The min. no. of final states in finite automaton are zero; and as maximum, all states of finite automata may be final states. The model of finite automaton can be represented graphically by



SIMPLIFIED NOTATIONS :-

Following are the ways to represent finite automaton.

⇒ Transition Equations

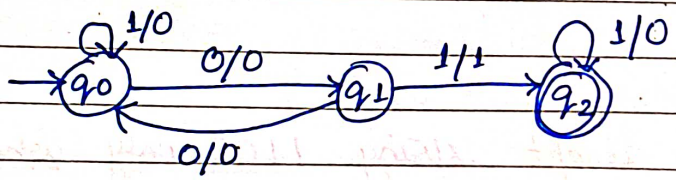
⇒ State Transition Graphs

⇒ Transition Table

STATE TRANSITION GRAPHS :-

→ A transition diagram (also called transition graph or transition system) is a finite directed labelled graph in which each vertex represents a state & directed edges indicate the transition from one state to another, edges are labelled with i/p or input/output.

→ In this representation, initial state is represented by a circle with an arrow towards it, final state is represented with two concentric circles, and intermediate states are just represented by a circle.



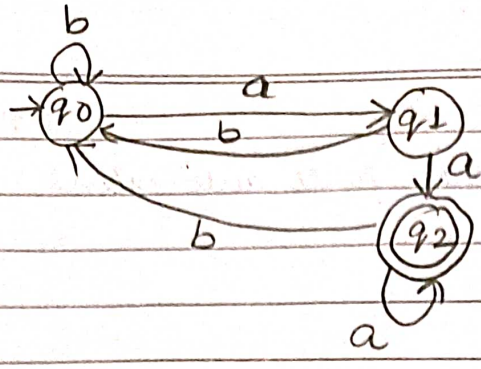
Transition graph were invented by John Myhill in 1957.

TRANSITION TABLE :-

→ A transition is the tabular representation of the transition system of an automaton. A transition table is also known as transition function table or state table.

→ In this representation initial state is represented by an arrow towards it, & final state is represented by a circle. Let us consider following transition table

PRESENT STATE	INPUT	
	0	1
→ q0	q1	q0
q1	q2	q0
(q2)	q2	q0



LANGUAGE OF DFA

→ The set of all strings that results in a sequence of state transition from start to an accepting state.

→ Now we can define the language of DFA $M = \{Q, \Sigma, \delta, q_0, F\}$

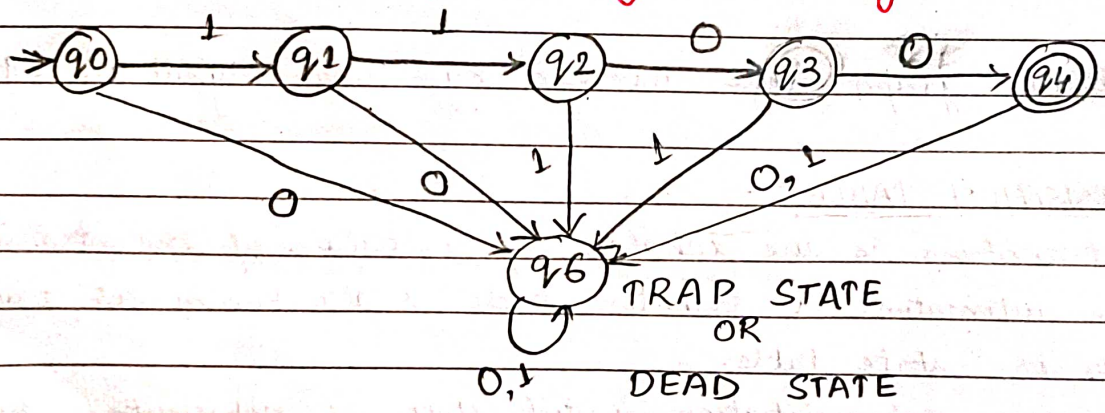
$$L(M) = \{w / \delta(q_0, w) \text{ is in } F\}$$

Language is denoted by $L(M)$

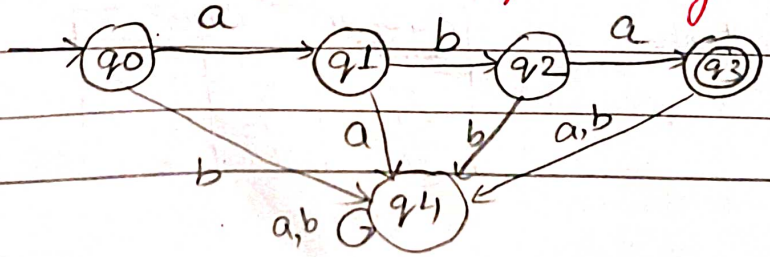
If L is $L(M)$ for some deterministic finite automata then we say L is a regular language.

Examples of DFA

① Design a DFA which accept string 1100 only where $\Sigma = \{0, 1\}$



② Design a DFA which accepts string aba.



PROPERTIES OF TRANSITION FUNCTIONS:-

① $\delta(q, \wedge) = q$ is a finite automaton. This means that state of system can be changed only by an input symbol.

② For all strings w & input symbols a,

$$\delta(q, aw) = \delta(\delta(q, a), w)$$
$$\delta(q, wa) = \delta(\delta(q, w), a)$$

Theorem

Prove that for any transition function δ & for any two input strings x and y

$$\delta(q, xy) = \delta(\delta(q, x), y)$$

Proof

Using mathematical induction we can prove this

when $|y| = 1$, $y = a \in \Sigma$

$$\delta(q, xy) = \delta(q, xa)$$
$$= \delta(\delta(q, x), a) \text{ using second property of transition function.}$$

Assume that the result is true for all strings x & string y with $|y| = n$. Let y be a string of length $n+1$.

$\therefore y = y_1 a$ where $|y_1| = n$

$$\delta(q, xy) = \delta(q, xy_1 a) = \delta(q, x_1 a) \qquad x_1 = xy_1$$
$$= \delta(\delta(q, x_1), a)$$
$$= \delta(\delta(q, xy_1), a)$$
$$= \delta(\delta(\delta(q, x), y_1), a)$$

$$\delta(q, xy) = \delta(\delta(q, x), y)$$
$$= \delta(\delta(q, x), y_1 a)$$
$$= \delta(\delta(\delta(q, x), y_1), a) = \delta(q, xy).$$

Hence proved.

ACCEPTABILITY OF A STRING BY A FINITE AUTOMATA:-

A string x is accepted by a finite automaton

$$M = (Q, \Sigma, \delta, q_0, F)$$

if $\delta(q_0, x) = q$ for some $q \in F$.

PROCESSING OF STRINGS BY DFA:-

Suppose $a_1, a_2, \ldots a_n$ is a sequence of input symbols. We start out with deterministic finite automata having $q_0, q_1, q_2, \ldots q_n$ states where q_0 is initial state & q_n is final state & transition function & processed as —

$$\delta(q_0, a_1) = q_1$$
$$\delta(q_1, a_2) = q_2$$
$$\delta(q_2, a_3) = q_3$$
$$\vdots \qquad \vdots$$
$$\delta(q_{n-1}, a_n) = q_n$$

Input $a_1, a_2, \ldots a_n$ is said to be "accepted" since q_n is member of final states, and if not then it is "rejected". By adopting this approach we can define the language of a particular DFA, and can also check that particular string is accepted or rejected by given FA.

FACTS IN DESIGNING PROCEDURE OF FA :-

a) In the designing of FA, first of all we have to analyse the set of strings i.e. language accepted by FA.

b) Make sure that every state is check for output state for every input symbol of Σ.

c) Make sure that no state must have two different output state for single input symbol.

d) Make sure, there is one initial state & at least one final state in transition diagram of FA.

Q1. Design FA that accepts set of strings such that every string ends in 00, over alphabet.

$\{0,1\}$ i.e. $\Sigma = \{0,1\}$

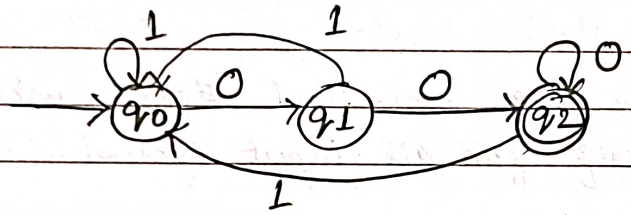
Sol.

$M = (Q, \Sigma, \delta, q_0, F)$ be the DFA

q_0 : initial state

Σ : $\{0,1\}$ is given

So according to the problem we have to design FA which accepts strings $w = w'00$ where w' — 0111100, 111100, 00, 100 ----.



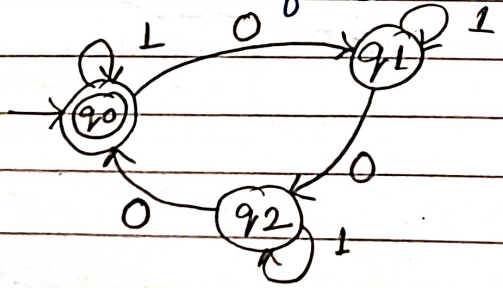
Q2. Construct FA that accepts set of strings where the no. of 0's in every string is multiple of three over alphabet $\Sigma = \{0,1$

Sol.

Multiple of 3 means no. of 0's in the strings may be 0,3,6, 12, ... in the no. at any position

Let FA be $M = (Q, \Sigma, \delta, q_0, F)$

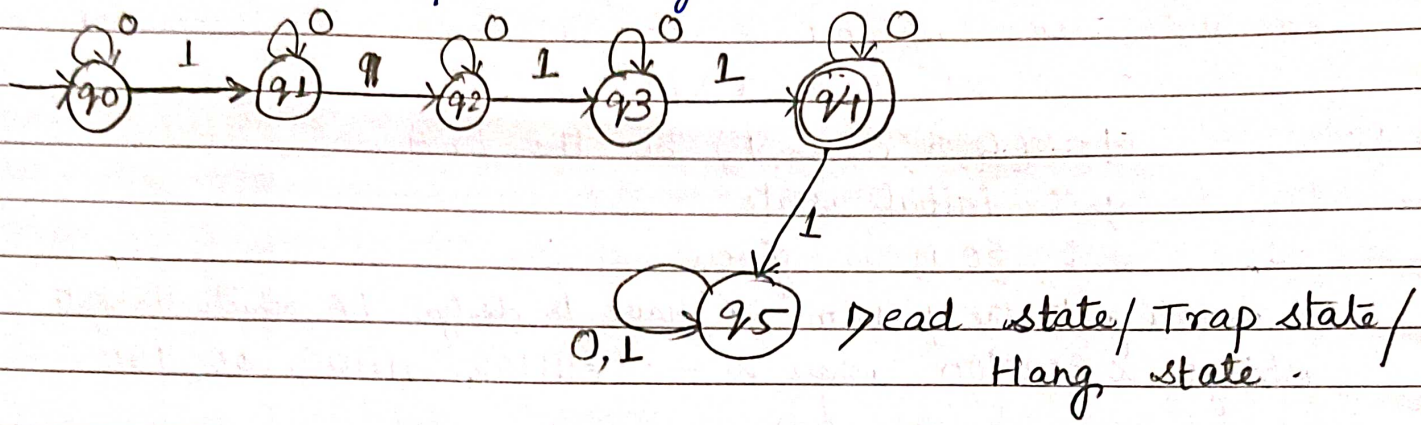
Here it is necessary to consider acceptance of Null (ϵ) string since it contains no. of 0's zero times.



Q3. Design a FA which accepts set of strings containing exactly four 1's in every string over alphabet $\Sigma = \{0,1\}$

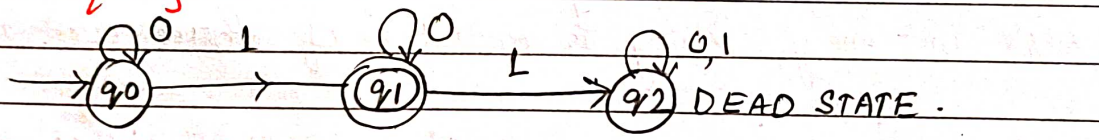
Sol:-

DFA should accepts the strings 1111, 01010111, 011010001 ----



Dead states :- These are non-final states which transit in itself for all input symbol.

Q4. Design a FA that accepts strings containing exactly 1 over alphabet $\{0,1\}$.



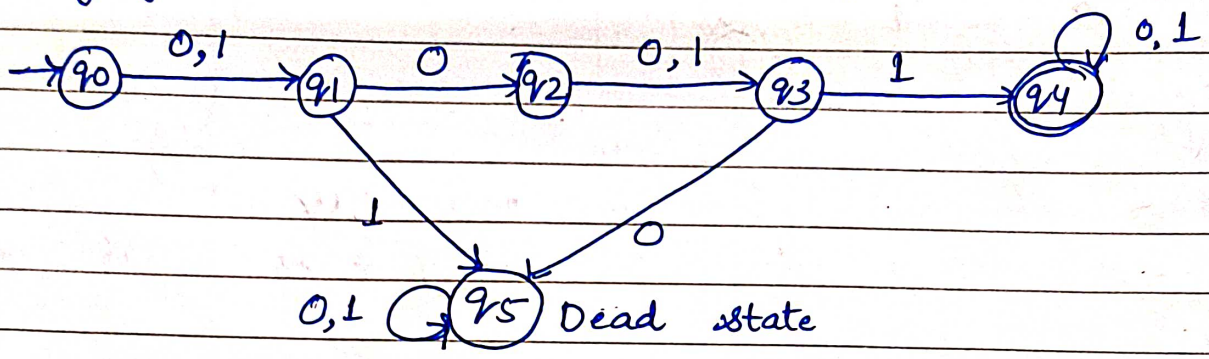
δ/Σ	0	1
→q0	q0	q1
*q1	q1	q2
q2	q2	q2

Non final state transit in itself

Q5. Design a FA which accepts the language $L = \{ w \in \{0,1\}^* \mid$ second symbol is '0' & fourth input is 1$\}$

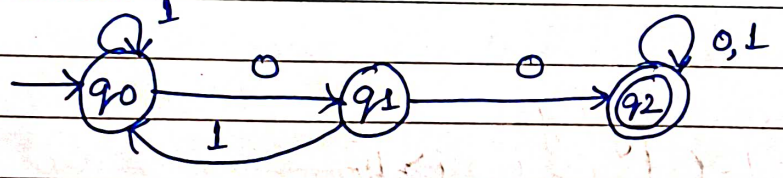
Sol.

After analysing the problem we find that every string of the language is like 0001, 1011, 10110000, ------.



Q6. For given FA write the language & also give transition table.

Sol.



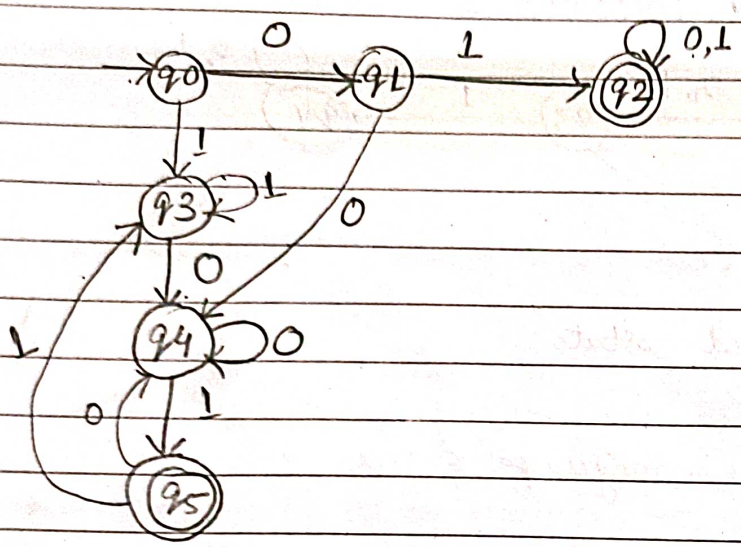
$L = \{ w \in (0,1)^* \mid$ every string w of the language containing 00 as substring $\}$.

δ/Σ	0	1
→q0	q1	q0
q1	q2	q0
*q2	q2	q2

Q7. Design a FA over alphabet $\Sigma = \{0,1\}$ which accepts set of strings either start with 01 or end with 01

Sol.

By the analysis of problem, it is clear that FA will accepts the strings such as 01, 01111, 000101, 00011101, ----

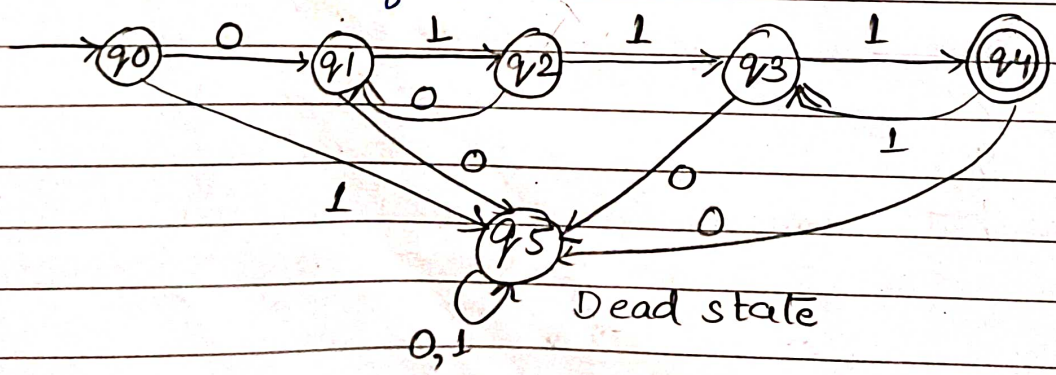


Q8. Design FA for the language

$$L = \{ (01)^i 1^{2j} \mid i \geq 1, j \geq 1 \}$$

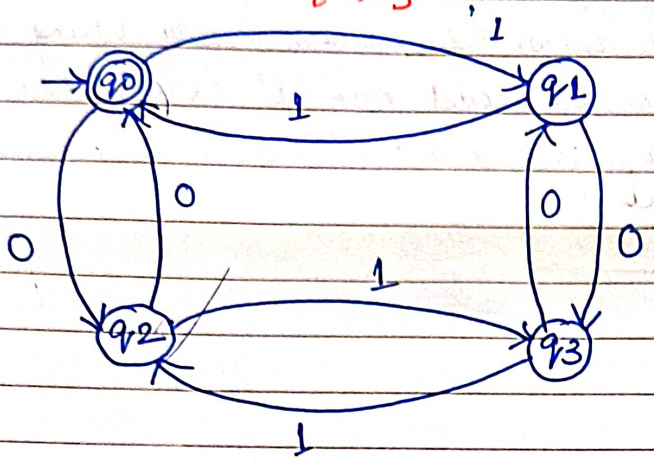
Sol.

By analysing language L, it is clear that FA will accepts strings start with with any no of 01 (not empty) and end with even no. of 1's.



Q9. Design a FA which accepts the language $L = \{ w / w$ has both an even no. of 0's and an even no. of 1's over alphabet $\Sigma = \{0, 1\}$.

Sol.



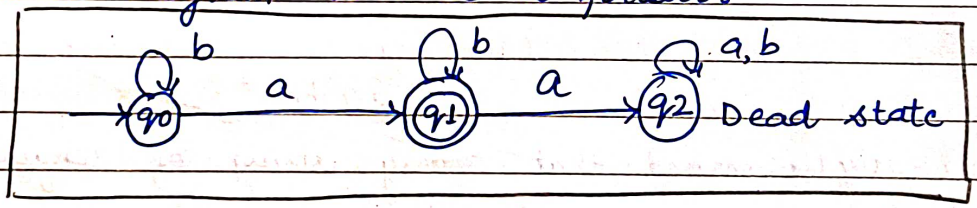
Q10. Design DFA for the language

$L = \{ w : n_a = 1 , w \in (a, b)^* \}$

Sol.

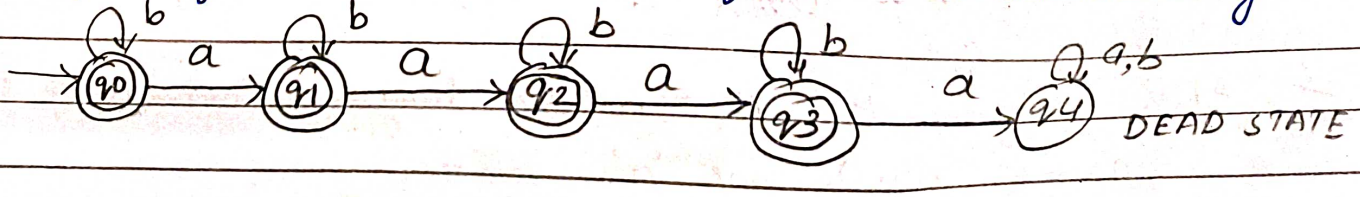
After analysing the language, it is clear that FA accepts string contains exactly one 'a' only that is a, bbb... a, bb... abbb,

Transition diagram will be as follows:-



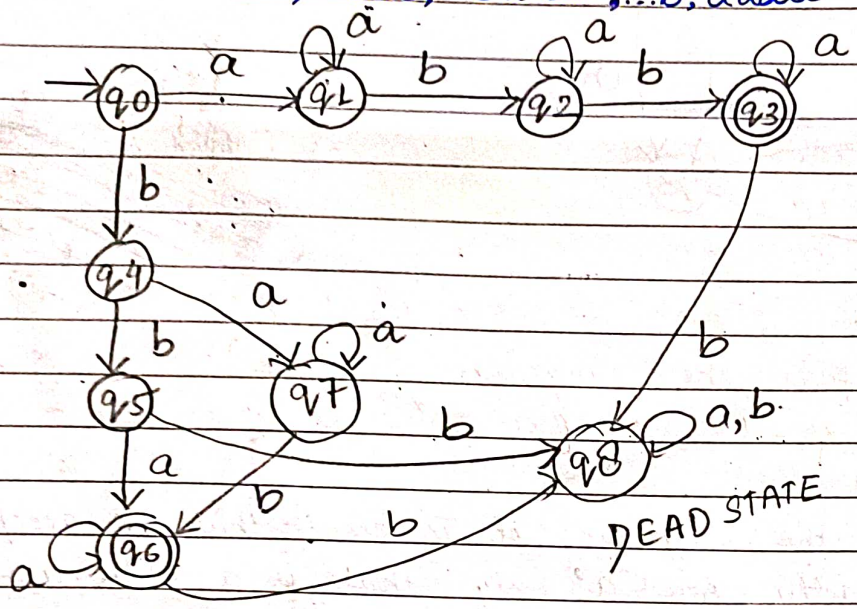
Q11. Design the deterministic finite automata for the given language $L = \{ w : n_a(w) \leq 3, w \in \{a, b\}^* \}$

Sol. Let us analyse the language first, clearly language L contains set of strings in which max. no. of a's can be three only.



Q12: Design DFA for given language $L = \{ w : n_a(w) \geq 1, n_b(w) = 2, w \in \{a,b\}^* \}$.

Sol.: Let us analyse the language first, since here conditions are defined as $n_a(w) \geq 1$ and $n_b(w) = 2$ means every string of the language must contain at least one 'a' exactly two b's such as abb, bab, baaaa, ...b, aaaa. ---.



Q13. Design DFA for the language $L = \{ w_1 ab w_2 : w_1, w_2 \in \{a,b\}^* \}$

Sol.: It is clearly observed that every string of language L has at least one ab in it at any position.

Q14. Give description of the language for DFA

Sol.: After analysing it, we can see language consists of string having atleast one 'a'

Q15. Consides finite state machine whose transition function δ is given by Table in form of transition table. Here $Q = \{q_0, q_1, q_2, q_3\}$ $\Sigma = \{0, 1\}$ $F = \{q_0\}$. Give the entire sequence of states for input string 11010

Table

δ / Σ	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

$\delta(q_0, 11010)$
$= \delta(q_1, 1010)$
$= \delta(q_0, 010)$
$= \delta(q_2, 10)$
$= \delta(q_3, 0)$
$= \delta(q_1, 1)$
$= q_0$ (Final state)

Q16: Prove that if $\delta(q, x) = \delta(q, y)$ then $\delta(q, xz) = \delta(q, yz)$

Soln:

$$\delta(q, xz) = \delta(\delta(q, x), z) = \delta(\delta(q, y), z) = \delta(q, yz)$$

Likewise $\delta(q, yz) = \delta(\delta(q, y), z) = \delta(\delta(q, x), z) = \delta(q, xz)$

Q17: For finite state machine M described by table

Q / Σ	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Find the strings among the following strings which are accepted by M:

a) 101101

$\delta(q_0, 101101) = \delta(q_0, 01101) = \delta(q_3, 1101)$
$= \delta(q_2, 101) = \delta(q_3, 01)$
$= \delta(q_1, 1)$
$= q_0$ (final state)

$\therefore$ Hence string is accepted.

b) 11111

$\delta(q_0, 11111) = \delta(q_1, 1111) = \delta(q_0, 111)$

$= \delta(q_1, 11)$

$= \delta(q_0, 1) = q_1$ (Non-final state)

Therefore, string is not accepted by given Finite state M/c.

c) 000000

$\delta(q_0, 000000) = \delta(\delta(q_0, 00000), 0)$

$= \delta[\delta(\delta(q_0, 0000), 0), 0)$

$= \delta(\delta(\delta(\delta(q_0, 000), 0), 0), 0)$

$= \delta(\delta(\delta(\delta(\delta(q_0, 00), 0), 0), 0), 0)$

$= \delta(\delta(\delta(\delta(\delta(\delta(q_0, 0), 0), 0), 0), 0), 0)$

$= \delta(\delta(\delta(\delta(\delta(q_2, 0), 0), 0), 0), 0)$

$= \delta(\delta(\delta(\delta(q_0, 0), 0), 0), 0)$

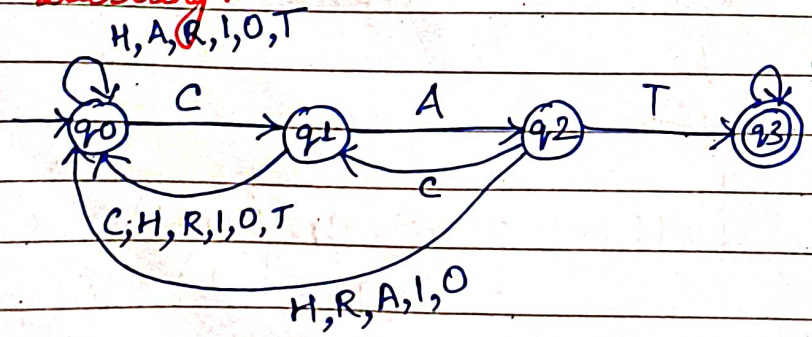
$= \delta(\delta(\delta(q_2, 0), 0), 0)$

$= \delta(\delta(q_0, 0), 0)$

$= \delta(q_2, 0) = q_0$ (Final state)

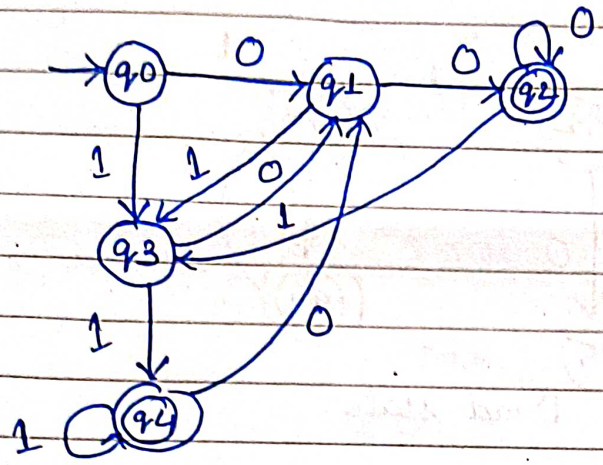
Hence 000000 is accepted by M

Q18. Design DFA that reads strings made up of letters in the 'CHARIOT' & recognize those strings that contain word 'CAT' as substring.



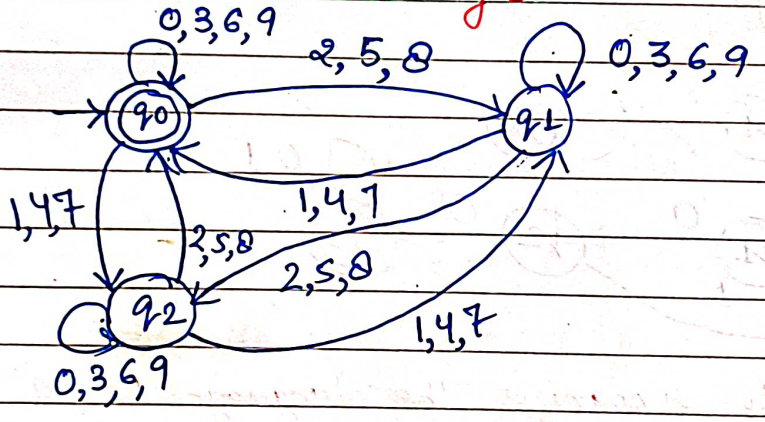
Q19. Design DFA that reads strings made up of {0,1} & accept only those strings which ends up in either 00 or 11.

Sol.y

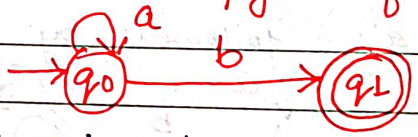


Q20. Construct DFA that accepts the set of natural numbers x which are divisible by 3.

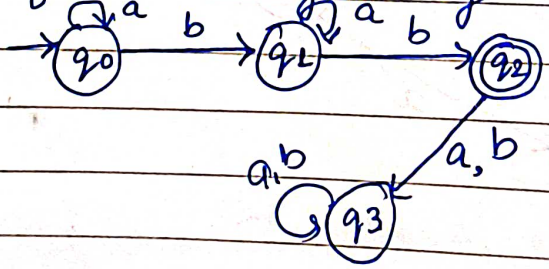
Sol.y



Q21. Here DFA is given in the figure for the language L. Find DFA for L^2.

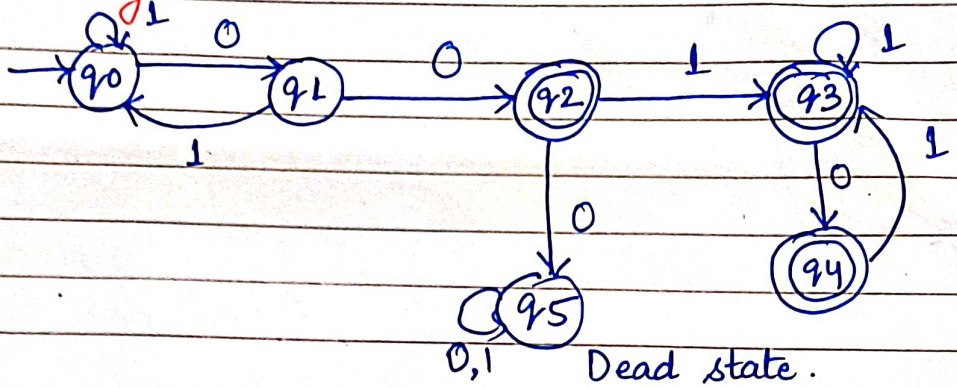


Sol.y DFA for L^2 is given by



Q.22 Design a DFA which accepts set of strings such that every string containing 00 as substring but not 000 as substring.

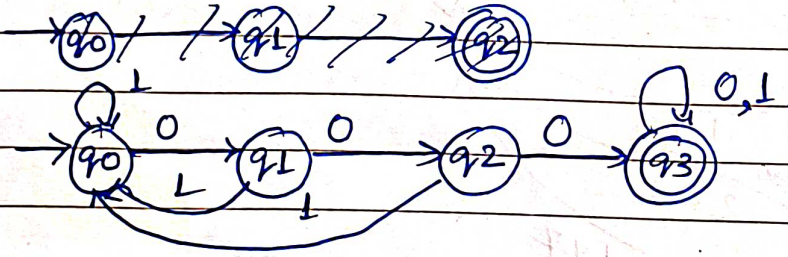
Sol.



Q23. Construct DFA that accepts strings on {0,1}

a) Set of all strings with three consecutive 0's.

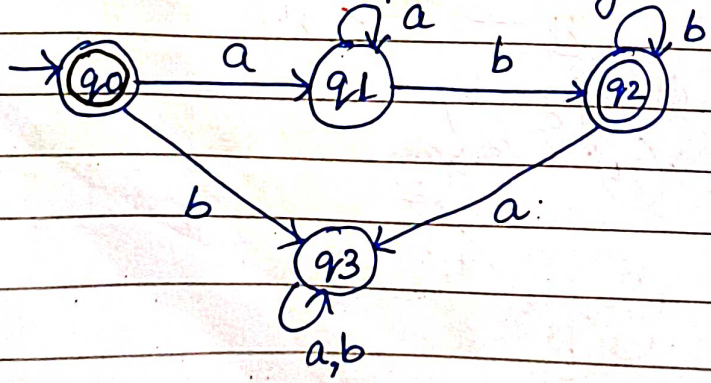
Sol.



Q24. Construct DFA which recognizes the language $L = \{a^m b^n : m, n > 0\}$

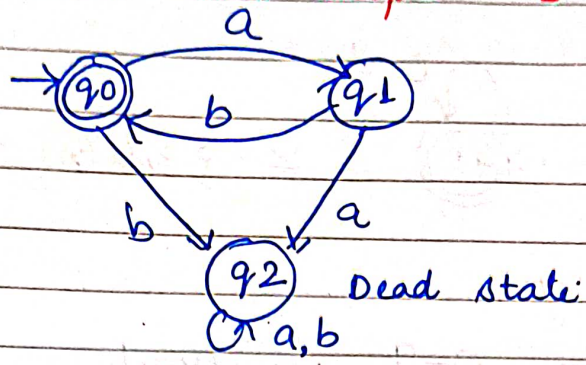
Sol.

Given language L has all strings which begin with one or more a's followed by one or more b's.



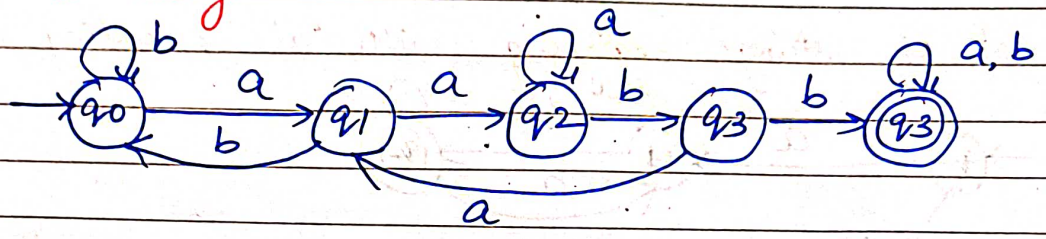
Q25. Design DFA that accepts $L = \{(ab)^n : n \geq 0\}$

Sol.n



Q26. Determine automata with set of strings having aabb as substring.

Sol.n



Q27. Design DFA which accepts set of all strings which are divisible by 3.

Sol.n

Remainder set = {0,1,2}

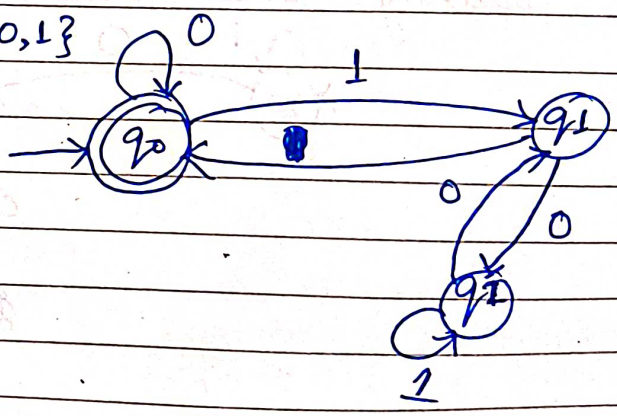
Consider $0 \to q_0$

$1 \to q_1$

$2 \to q_2$

$\Sigma = \{0,1\}$

$\delta(q, a) = 2 \times q + \Sigma$



$\delta(q_0,0) = 2\times0+0 = 0 = q_0$

$\delta(q_0,1) = 2\times0+1 = 1 = q_1$

$\delta(q_1,0) = 2\times1+0 = 2 = q_2$

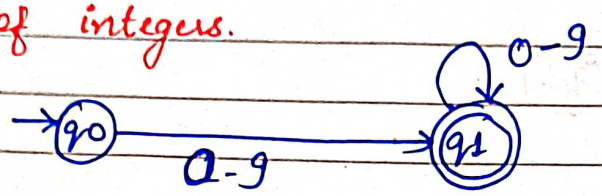
$\delta(q_1,1) = 2\times1+1 = 3 \bmod 3 = 0\ q_0$

$\delta(q_2,0) = 2\times2+0 = 4 \bmod 3 = 1\ q_1$

$\delta(q_2,1) = 2\times2+1 = 5 \bmod 3 = 2\ q_2$

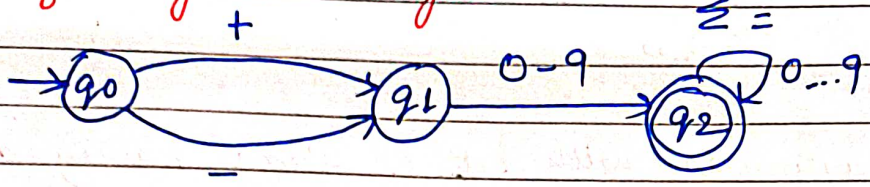
Q28. Determine DFA with the

a) set of integers.

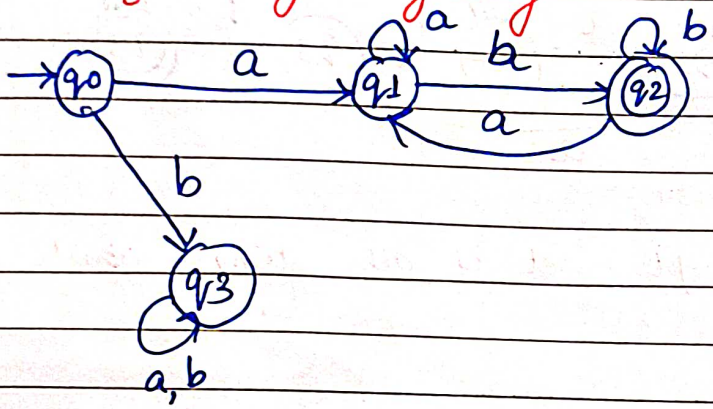


$\Sigma = \{0, 1, 2, \ldots 9\}$

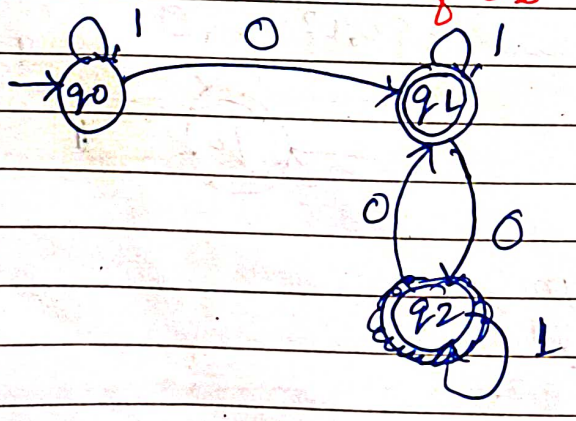
b) set of signed integers



c) Set of strings beginning with 'a' & ending with 'b'.

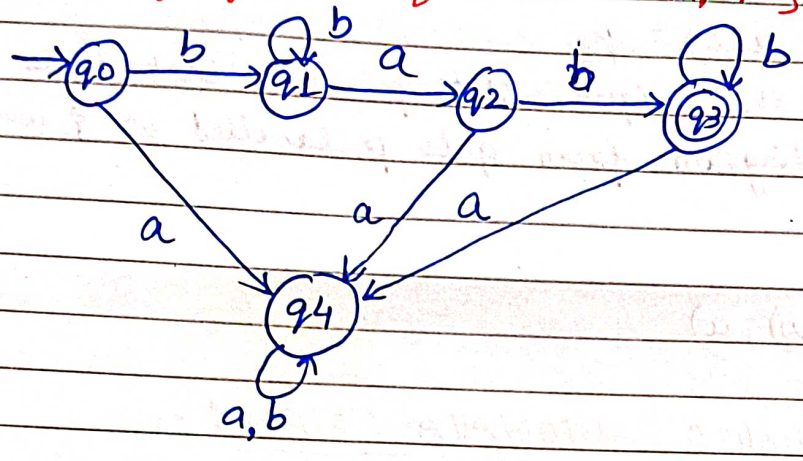


d) having odd number of 0's

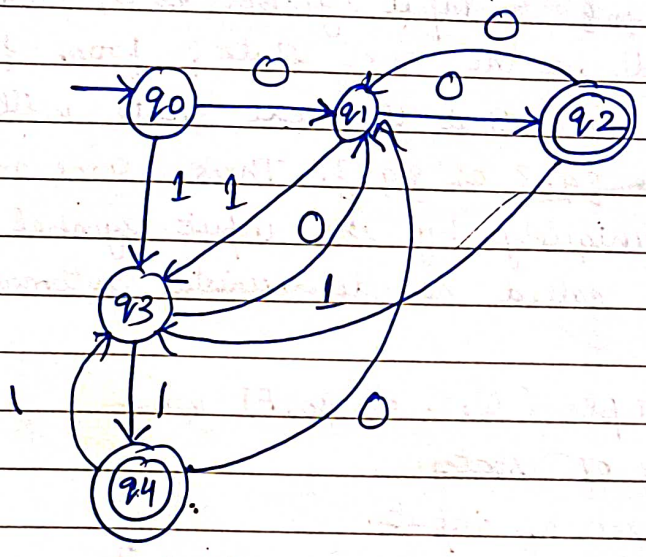


Q29. Given $\Sigma = \{a, b\}$ construct DFA that shall recognize the language $L = \{b^m a b^n : m, n > 0\}$

Sol:-



Q30. Draw a DFA that accepts set of all string of 0 and 1 that end in last two same symbols.



EXTENDED TRANSITION FUNCTION :-

The transition function $\delta : Q \times \Sigma \to Q$ can be extended to the function $\hat{\delta}$ or δ^* as $\hat{\delta} : Q \times \Sigma^* \to Q$.

Σ^* is a string rather than a single letter & its value gives the state automation will be in after reading the string.

For example, $\delta(q_0, a) = q_1$ $\quad$ $\delta(q_2, c) = q_3$

$\delta(q_1, b) = q_2$

$\delta^*(q_0, abc) = q_3$

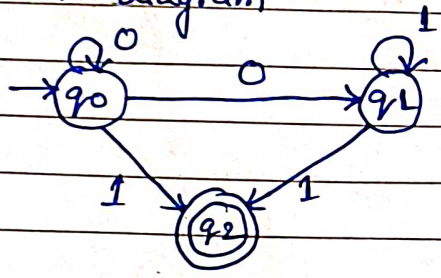
i.e. $\hat{\delta}(q, w)$ or $\delta^*(q, w)$ is the unique state p such that there is a path in the transition diagram from q to p labelled w & we define :

i) $\hat{\delta}(q, \Lambda) = q$

ii) $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$

NON- DETERMINISTIC FINITE AUTOMATA (NFA) :-

We explain the concept of non-deterministic finite automata using a transition diagram



If the automaton is in a state $\{q_0\}$ & input symbol is 0, what will be the next state ? from fig. it is clear that the next state will be either $\{q_0\}$ or $\{q_1\}$. Thus some moves of the m/c cannot determined uniquely by the input symbol & the present state. Such machine are called non-deterministic automata.

Definition :- A NDFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where

i) Q is a finite non-empty set of states.

ii) Σ is a finite non-empty set of inputs.

iii) δ is the transition function mapping $Q \times \Sigma$ into 2^Q, set of all subset of Q : $Q \times \Sigma \to 2^Q$

iv) $q_0 \in Q$ is the initial state.

v) $F \subseteq Q$ is the set of final state.

Characteristics of NFA:-

i) One i/p symbol causes to move more than one state or a state may contain more than one transition from same input symbol.

ii) It is not compulsory that all the states have to consume all the I/p symbols represent in Σ.

Language Accepted by NDFA:-

The language L accepted by NFA $M = (Q, \Sigma, \delta, q_0, F)$ is defined as the set of all string accepted in the above sense. Formally,

$$L(M) = \{ w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \emptyset \}$$

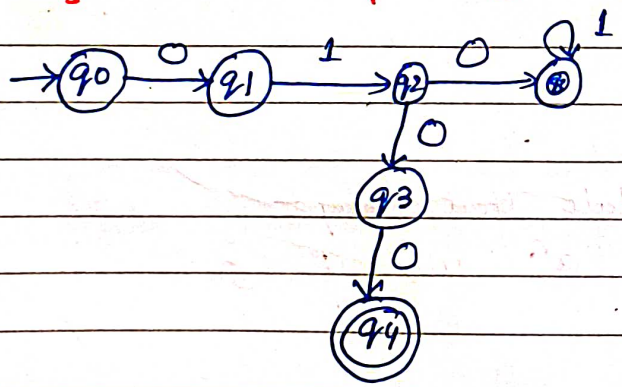
In other words language consists of all string w for which there is a walk labelled w from initial vertex of transition graph to final vertex.

Examples on NFA

Q1 Construct NFA for the language :-

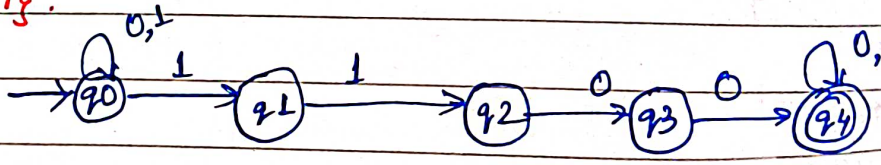
$L = \{ 0101^n \cup 0100 \mid n \geq 0 \}$

Sol.ⁿ



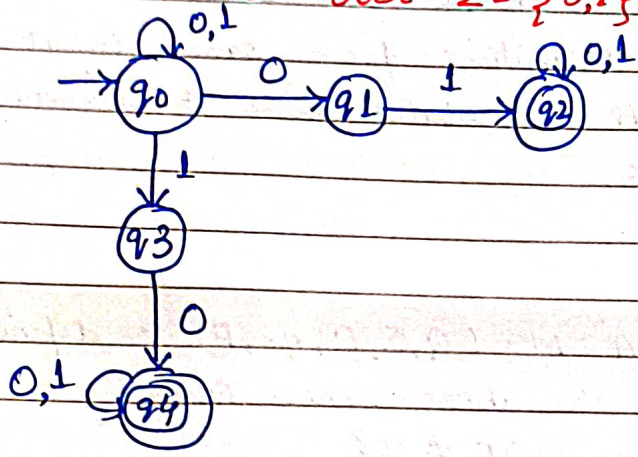
Q2. Construct NFA in which double '1' is followed by double '0' over $\Sigma = \{0, 1\}$.

Sol.ⁿ



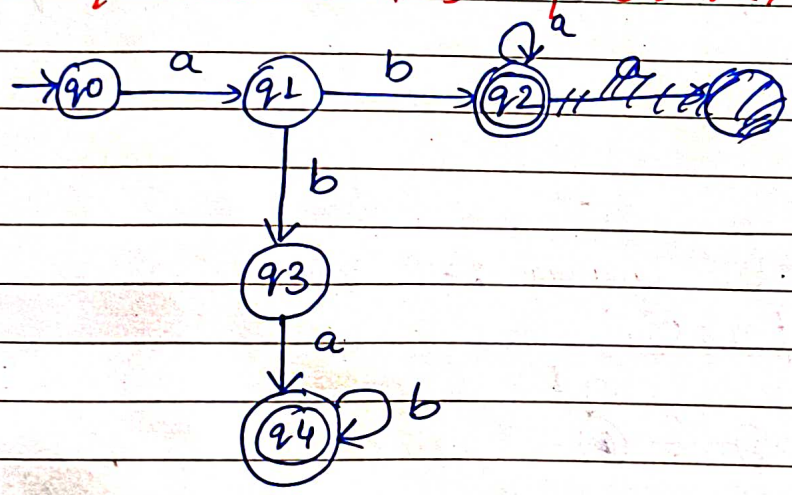
Q3. Design NFA which accepts the string containing either '01' or '10' over $\Sigma = \{0,1\}$

Sol.



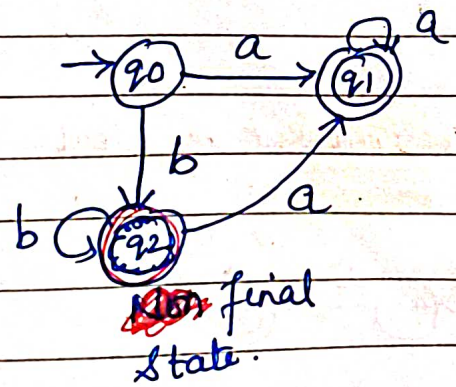
Q4. Design NFA with no more than five states for set $\{abab^n : n>0\} \cup \{aba^n : n>0\}$

Sol.



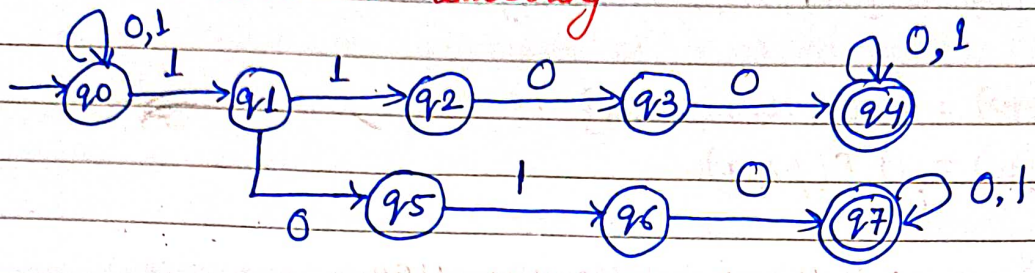
Q5. Design NFA with 3 state that accept $L = \{a^n : n \geq 1\} \cup \{b^m a^k : m \geq 0, k \geq 0\}$

Sol.



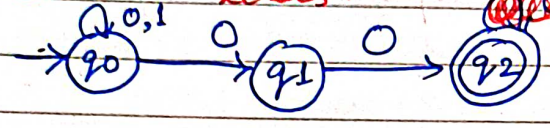
Q6. Design NFA which accepts set of all binary strings containing 1100 or 1010 as substring.

Sol.:



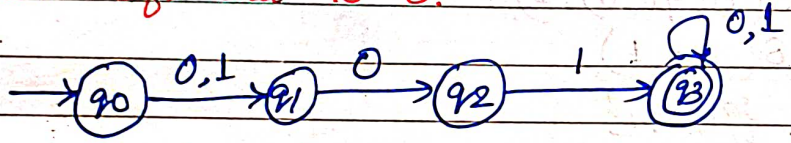
Q7: Design NFA which accepts set of strings ending with 2 consecutive zeroes.

Sol.:



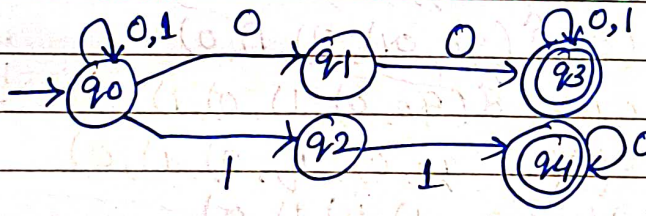
Q8: Design NFA in which set of all binary strings containing the third symbol from left end is 1 and second symbol from left end is 0.

Sol.:



Q9. Design NFA for language L = all strings over $\Sigma = \{0,1\}$ that have at least two consecutive 0's & 1's.

Sol.:



EXTENDED TRANSITION FUNCTION FOR NFA:-

We define extended transition function ($\hat{\delta}$) for NFA's transition function δ by:

Basis: $\hat{\delta}(q, \epsilon) = \{q\}$ and $w \in \Sigma^*$

Induction: Suppose w is of the form $w = xa$, where a is last symbol of w and x is remaining string of w.

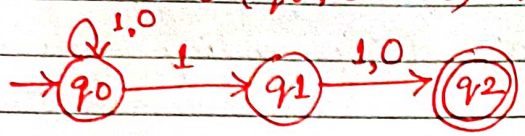
Also suppose that $\hat{\delta}(q,x) = \{p_1, p_2, \ldots p_k\}$

Let $\bigcup_{i=1}^{k} \delta(p_i, a) = \{r_1, r_2, \ldots, r_m\}$

then $\hat{\delta}(q,w) = \{r_1, r_2 \ldots r_m\}$ i.e.

$$\hat{\delta}(q,w) = \bigcup_{i=1}^{k} \delta(p_i, a)$$

Example Find $\hat{\delta}(q_0, 01010)$ in following NFA



which accepts all strings w whose second last symbol is 1.

Soln

$$\hat{\delta}(q_0, \epsilon) = \{q_0\}$$

$$\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0\}$$

$$\hat{\delta}(q_0, 01) = \delta(\hat{\delta}(q_0, 0), 1)$$
$$= \delta(\delta(q_0, 0), 1)$$
$$= \delta(q_0, 1)$$
$$= \{q_0, q_1\}$$

$$\hat{\delta}(q_0, 01010) = \delta(\hat{\delta}(q_0, 0101), 0)$$
$$= \delta(\hat{\delta}(\hat{\delta}(q_0, 010), 1), 0)$$
$$= \delta(\delta(\delta(\hat{\delta}(q_0, 01), 0), 1), 0)$$
$$= \delta(\delta(\delta(\delta(\hat{\delta}(q_0, 0), 1), 0), 1), 0)$$
$$= \delta(\delta(\delta(\delta(\delta(q_0, 0), 1), 0), 1), 0)$$
$$= \delta(\delta(\delta(\delta(q_0, 1), 0), 1), 0)$$
$$= \delta(\delta(\delta(\{q_0, q_1\}, 0), 1), 0)$$
$$= \delta(\delta(\{q_0, q_2\}, 1), 0)$$
$$= \delta(\{q_0, q_1\}, 0)$$
$$= \{q_0, q_2\}.$$

Since q_2 is final state.

$\therefore$ w is accepted by given NFA.

NFA with ϵ- transitions :-

NFA with ϵ- transitions can change their state on empty input string that is without any input or ϵ as an input.

Definition :-

NFA with ϵ-moves is denoted by 5-tuple notation

$$M = (Q, \Sigma, \delta, q_0, F) \text{ where}$$

Q — is a finite set of states

Σ — is a finite set of i/p alphabet

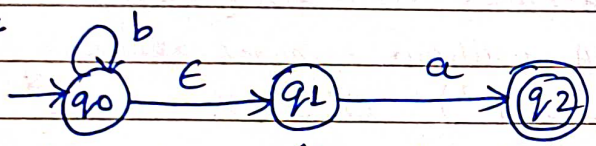
q_0 — is a initial state.

δ — is a transition function which maps

$$Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$$

F — is a finite set final state.

Example



The meaning of epsilon transition is transition on empty input of length 0. In the above figure without regarding any i/p symbol the transition moves from state q_0 to q_1.

Any FA with ϵ- transitions is always a NFA with ϵ-moves.

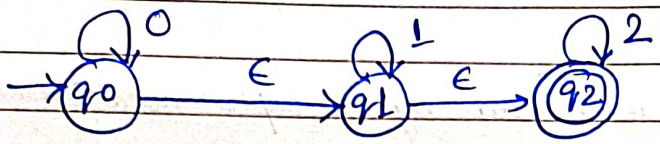
Use of ϵ-moves

If we want to construct a finite automata which accept language, it becomes very difficult or many times it seems to impossible to draw directly NFA or DFA. If you go for NFA with ϵ-moves and DFA or NFA without ϵ-moves acceptance of same language can be described.

ϵ- closure :-

ϵ-closure of q is pronounced as epsilon closure of q_0 is defined as the set of all states p such that there is a path from q_0 to p with label ϵ.

Example



ϵ-closure $(q_0) = \{q_0, q_1, q_2\}$
ϵ- closure $(q_1) = \{q_1, q_2\}$
ϵ- closure $(q_2) = \{q_2\}$

Construction of NFA without ϵ-moves from NFA with ϵ-moves.

If NFA with ϵ-moves is given by

$$M_1 = (Q, \Sigma, \delta, q_0, F) \qquad \delta : Q \times (\Sigma \cup \epsilon) \to 2^Q$$

It can be converted into NFA without ϵ-moves

$$M_2 = (Q, \Sigma, \delta', q_0, F') \qquad \delta' : Q \times \Sigma \to 2^Q$$

Q, Σ, initial state q_0 are same.

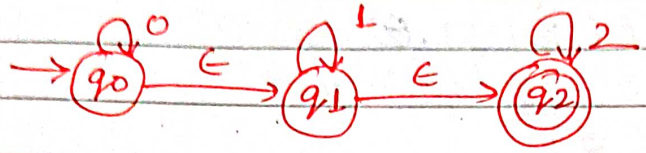
Method

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA with ϵ-Moves. In order to construct equivalent NFA without ϵ-moves $M' = (Q, \Sigma, \delta', q_0, F')$ we follow the following steps:-

i) $F' = F \cup \{q\}$
if ϵ-closure (q) contains member of F or $F = F'$ otherwise

ii)
$$\delta'(q, a) = \epsilon\text{-closure}(\delta(\hat{\delta}(q, \epsilon), a))$$
$$\hat{\delta}(q, \epsilon) = \epsilon\text{-closure}(q)$$

Example Convert NFA with ϵ-moves in figure to equivalent NFA without ϵ-moves.



Sol. First find ϵ-closure of all states

ϵ-closure (q0) = {q0, q1, q2}

ϵ-closure (q1) = {q1, q2}

ϵ-closure (q2) = {q2}

q0, ϵ
q0, 0
q0, ϵ

Now, we shall find out the transition function δ'

$\delta'(q_0, 0)$ = ϵ-closure ($\delta(\hat{\delta}(q_0, \epsilon), 0)$)

= ~~ϵ-closure ($\delta(q_0, 0)$)~~ ϵ-closure (δ(ϵ-closure (q_0), 0))

= ϵ-closure (δ ({q_0, q_1, q_2}, 0))

= ϵ-closure (q_0)

$\delta'(q_0, 0) = \{q_0, q_1, q_2\}$

$\delta'(q_0, 1)$ = ϵ-closure ($\delta(\hat{\delta}(q_0, \epsilon), 1)$)

= ϵ-closure (δ(ϵ-closure (q_0), 1))

= ϵ-closure (δ ({q_0, q_1, q_2), 1)

= ϵ-closure (q_1)

$\delta'(q_0, 1) = \{q_1, q_2\}$

$\delta'(q_0, 2)$ = ϵ-closure ($\delta(\hat{\delta}(q_0, \epsilon), 2)$)

= ϵ-closure (δ(ϵclosure (q_0), 2))

= ϵ-closure (δ({q_0, q_1, q_2}, 2))

= ϵ-closure (q_2)

$\delta'(q_0, 2) = \{q_2\}$

Similarly we can obtain

$\delta'(q_1, 0) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_1, \epsilon), 0))$
$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 0))$
$= \epsilon\text{-closure}(\emptyset)$

$\boxed{\delta'(q_1, 0) = \emptyset}$

$\delta'(q_1, 1) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_1, \epsilon), 1))$
$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 1))$
$= \epsilon\text{-closure}(q_1)$

$\boxed{\delta'(q_1, 1) = \{q_1, q_2\}}$

$\delta'(q_1, 2) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_1, \epsilon), 2)$
$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 2)$
$= \epsilon\text{-closure}(q_2) =$

$\boxed{\delta'(q_1, 2) = \{q_2\}}$

$\delta'(q_2, 0) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_2, \epsilon), 0))$
$= \epsilon\text{-closure}(\delta(\{q_2\}, 0))$
$= \epsilon\text{-closure}(\emptyset)$

$\boxed{\delta'(q_2, 0) = \emptyset}$

$\delta'(q_2, 1) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_2, \epsilon), 1))$
$= \epsilon\text{-closure}(\delta(\{q_2\}, 1))$
$= \epsilon\text{-closure}(\emptyset)$

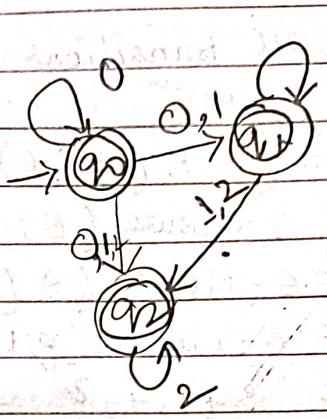
$\boxed{\delta'(q_2, 1) = \emptyset}$

$\delta'(q_2, 2) = \epsilon\text{-closure}(\delta(\hat{\delta}(q_2, \epsilon), 2))$
$= \epsilon\text{-closure}(\delta(\{q_2\}, 2))$
$= \epsilon\text{-closure}(q_2)$

$\boxed{\delta'(q_2, 2) = \{q_2\}}$

Now we can construct transition table for NFA without ε-moves as

State	Input 0	1	2
→ q0	{q0, q1, q2}	{q1, q2}	{q2}
q1	∅	{q1, q2}	{q2}
q2	∅	∅	{q2}



Construction of DFA with ε-moves :-

There are 2 approaches for constructing DFA from given NFA with ε-moves.

First method

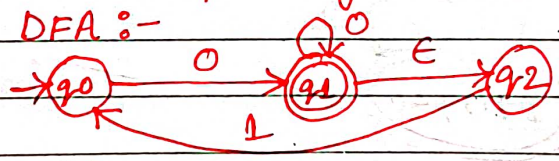
This consists of 2 steps :-

1. Construct NFA without ε-moves then
2. Construct an equivalent DFA from this newly constructed NFA without ε-moves.

Second method

In this approach we can directly start with initial state & go on adding states as and when required to the diagram till we come to stage when there exists no state without transitions specified.

Example Consider the following NFA with ε-moves and construct its equivalent DFA :-



Sol.^y To construct DFA

ε-closure (q0) = {q0}

ε-closure (q1) = {q1, q2}

ε-closure (q2) = {q2}

We define ϵ-closure(q_0) as initial state i.e. $\{q_0\}$ state

Now, we will transitions for all states.

$$\delta'(\{q_0\}, 0) = \hat{\delta}(\delta(\hat{\delta}(\{q_0\}, \epsilon), 0)$$

$\hat{\delta}$: ϵ-closure

$$= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(\{q_0\}), 0))$$
$$= \epsilon\text{-closure}(\delta(\{q_0\}, 0))$$
$$= \epsilon\text{-closure}(q_1)$$

$$\boxed{\delta'(\{q_0\}, 0) = \{q_1, q_2\}}$$ ——— New state

$$\delta'(\{q_0\}, 1) = \epsilon\text{-closure}(\delta(\hat{\delta}(\{q_0\}, \epsilon), 1))$$
$$= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(\{q_0\}), 1))$$
$$= \epsilon\text{-closure}(\delta(\{q_0\}, 1))$$
$$= \epsilon\text{-closure}(\emptyset)$$

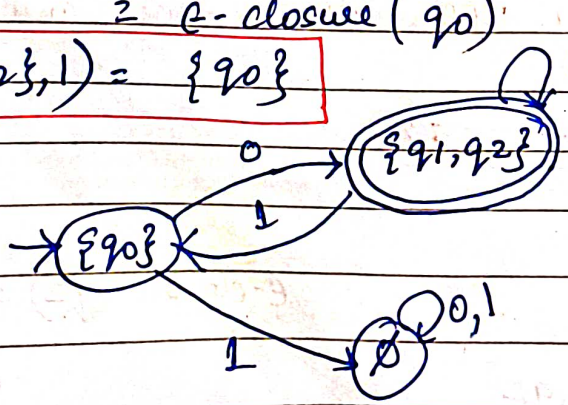
$$\boxed{\delta'(\{q_0\}, 1) = \emptyset}$$ ——— New state

$$\delta'(\{q_1, q_2\}, 0) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(\{q_1, q_2\}), 0))$$
$$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 0))$$
$$= \epsilon\text{-closure}(q_1)$$

$$\boxed{\delta'(\{q_1, q_2\}, 0) = \{q_1, q_2\}}$$

$$\delta'(\{q_1, q_2\}, 1) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(\{q_1, q_2\}), 1))$$
$$= \epsilon\text{-closure}(\delta(\{q_1, q_2\}, 1))$$
$$= \epsilon\text{-closure}(q_0)$$

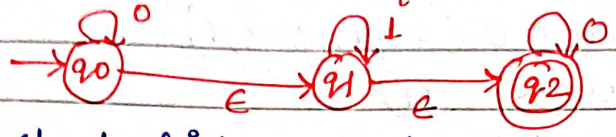
$$\boxed{\delta'(\{q_1, q_2\}, 1) = \{q_0\}}$$



	0	1
→{q0}	{q1,q2}	∅
{q1,q2}	{q1,q2}	q0
∅	∅	∅

Example

Construct DFA equivalent to following NFA



Sol.ⁿ To construct first we find ε-closure of all states

ε-closure $(q_0) = \{q_0, q_1, q_2\}$

ε-closure $(q_1) = \{q_1, q_2\}$

ε-closure $(q_2) = \{q_2\}$

we define ε-closure (q_0) as initial state i.e. $\{q_0, q_1, q_2\}$ is initial state.

Next we will define transitions for initial state & all new states.

$\delta'(\{q_0,q_1,q_2\}, 0) =$ ε-closure $(\delta($ εclosure $(\{q_0,q_1,q_2\}), 0))$

$=$ ε-closure $(\delta(\{q_0,q_1,q_2\}, 0))$

$=$ ε-closure $(\{q_0, q_2\})$

$\boxed{\delta'(\{q_0,q_1,q_2\}, 0) = \{q_0,q_1,q_2\}}$

$\delta'(\{q_0,q_1,q_2\}, 1) =$ ε-closure $(\delta($ εclosure $(\{q_0,q_1,q_2\}, 1))$

$=$ ε-closure $(\delta(\{q_0,q_1,q_2\}, 1))$

$=$ ε-closure $(\{q_1\})$

$\boxed{\delta'(\{q_0,q_1,q_2\}, 1) = \{q_1,q_2\}}$ ——— New state

$\delta'(\{q_1,q_2\}, 0) =$ ε-closure $(\delta($ εclosure $(\{q_1,q_2\}), 0))$

$=$ ε-closure $(\delta(\{q_1,q_2\}, 0))$

$=$ ε-closure $(\{q_2\})$

$\boxed{\delta'(\{q_1,q_2\}, 0) = \{q_2\}}$ ——— New state

$\delta'(\{q_1,q_2\},1) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{q_1,q_2\}),1))$

$= \epsilon\text{-closure}(\delta(\{q_1,q_2\},1))$

$= \epsilon\text{-closure}(q_1)$

$\delta'(\{q_1,q_2\},1) = \{q_1,q_2\}$

$\delta'(\{q_2\},0) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{q_2\},0))$

$= \epsilon\text{-closure}(\delta(\{q_2\},0))$

$= \epsilon\text{-closure}(\{q_2\})$

$= \{q_2\}.$

$\delta'(\{q_2\},1) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{q_2\}),1))$

$= \epsilon\text{-closure}(\delta(\{q_2\},1))$

$= \epsilon\text{-closure}(\emptyset)$

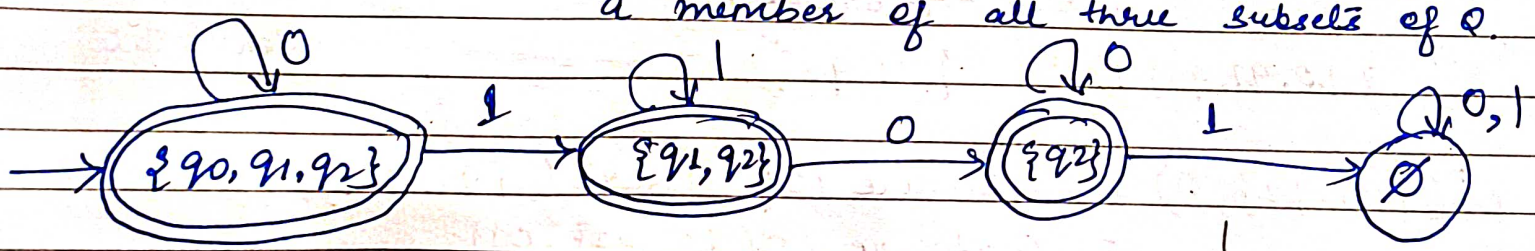
$\delta'(\{q_2\},1) = \emptyset$ ——— New state

$\delta'(\emptyset,0) = \emptyset$

$\delta'(\emptyset,1) = \emptyset$

Initial state = $\{q_0,q_1,q_2\}$

Final state = $\{q_0,q_1,q_2\}$, $\{q_1,q_2\}$, $\{q_2\}$. since q_2 is a member of all three subsets of Q.

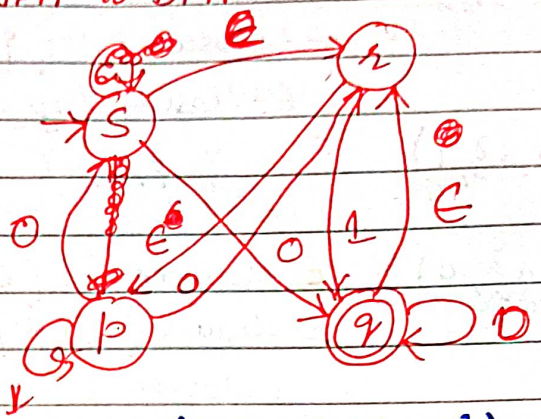


	0	1
$\{q_0,q_1,q_2\}$	$\{q_0,q_1,q_2\}$	$\{q_1,q_2\}$

Example

Convert the following ϵ-NFA to DFA

	ϵ	0	1
→p	∅	r,s	p
*q	r	q	∅
r	p	∅	q
→s	r	q	∅



Sol. Let equivalent DFA be $D = (Q', \Sigma, \delta', q_0', F')$

ϵ-closure(s) = {s, r, p}
ϵ-closure(p) = {p}
ϵ-closure(q) = {q, r, p}
ϵ-closure(r) = {r, p}

We will define ϵ-closure(s) as initial state i.e. {s, r, p} is initial state

$\delta'(\{s,r,p\}, 0)$ = ϵ-closure(δ(ϵclosure({s,r,p}), 0))
= ϵ-closure(δ({s,r,p}, 0))
= ϵ-closure({q, r, s})
$\delta'(\{s,r,p\}, 0)$ = {p, q, r, s} ——— New state.

$\delta'(\{s,r,p\}, 1)$ = ϵ-closure(δ(ϵclosure({s,r,p}), 1))
= ϵ-closure(δ({s,r,p}, 1))
= ϵ-closure({p, q}) — New state
$\delta'(\{s,r,p\}, 1)$ = {p, q, r}

$\delta'(\{p,q,r,s\}, 0)$ = ϵ-closure(δ(ϵclosure({p,q,r,s}), 0)
= ϵ-closure(δ({p,q,r,s}, 0))
= ϵ-closure(q, r, s}
$\delta'(\{p,q,r,s\}, 0)$ = {p, q, r, s} ——— Old state

$\delta'(\{p,q,r,s\},1) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{p,q,r,s\}),1))$

$= \epsilon\text{-closure}(\delta(\{p,q,r,s\},1))$

$= \epsilon\text{-closure}(\{p,q\})$

$\delta'(\{p,q,r,s\},1) = \{p,q,r\}$

$\delta'(\{p,q,r\},0) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{p,q,r\},0))$

$= \epsilon\text{-closure}(\delta(\{p,q,r\},0))$

$= \epsilon\text{-closure}(\{q,r,s\})$

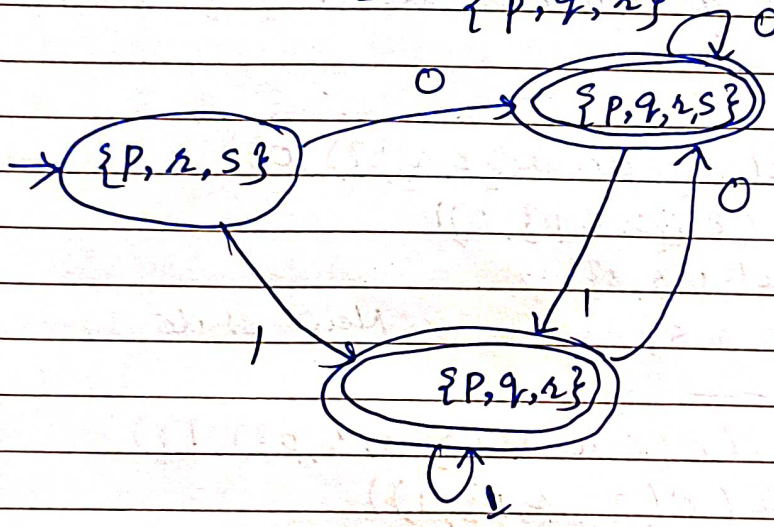
$\delta'(\{p,q,r\},0) = \{p,q,r,s\}$

$\delta'(\{p,q,r\},1) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{p,q,r\}),1))$

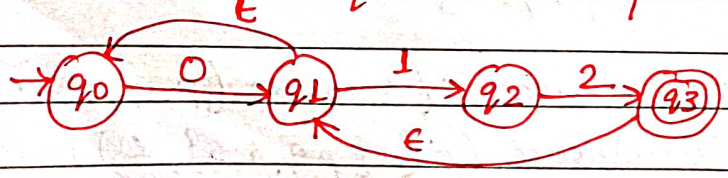
$= \epsilon\text{-closure}(\delta(\{p,q,r\},1))$

$= \epsilon\text{-closure}(\{p,q\})$

$= \{p,q,r\}$



Example Construct DFA equivalent to following NFA



Sol.

ϵ-closure of $(q_0) = \{q_0\}$ ϵ-closure$(q_2) = \{q_2\}$

ϵ-closure of $(q_1) = \{q_1, q_0\}$ ϵ-closure$(q_3) = \{q_1, q_3\}$

We will now define ϵ-closure(q_0) as initial state of DFA i.e. $\{q_0\}$

$\delta'(\{q_0\}, 0) = \epsilon$-closure($\delta$($\epsilon$closure$\{q_0\}$, 0))
$= \epsilon$-closure(q_1)
$= \{q_0, q_1\}$ ——— New state

$\delta'(\{q_0\}, 1) = \epsilon$-closure($\delta(\{q_0\}, 1)$)
$= \epsilon$-closure($\emptyset$)
$= \emptyset$ ——— Dead state.

$\delta'(\{q_0\}, 2) = \emptyset$

$\delta'(\{q_0, q_1\}, 0) = \epsilon$closure($\delta$($\epsilon$closure($\{q_0, q_1\}$), 0))
$= \epsilon$-closure($\delta(\{q_0, q_1\}, 0)$)
$= \epsilon$-closure(q_1)
$= \{q_0, q_1\}$.

$\delta'(\{q_0, q_1\}, 1) = \epsilon$-closure($\delta$($\epsilon$closure($\{q_0, q_1\}$, 1))
$= \epsilon$-closure($\delta(\{q_0, q_1\}, 1)$)
$= \epsilon$-closure(q_2)
$= \{q_2\}$ ——— New state

$\delta'(\{q_0, q_1\}, 2) = \emptyset$

$\delta'(\{q_2\}, 0) = \epsilon$closure($\delta$($\epsilon$closure($\{q_2\}$, 0))
$= \epsilon$closure($\delta(\{q_2\}, 0)$)
$= \epsilon$closure($\emptyset$) $= \emptyset$

$\delta'(\{q_2\}, 1) = \emptyset$

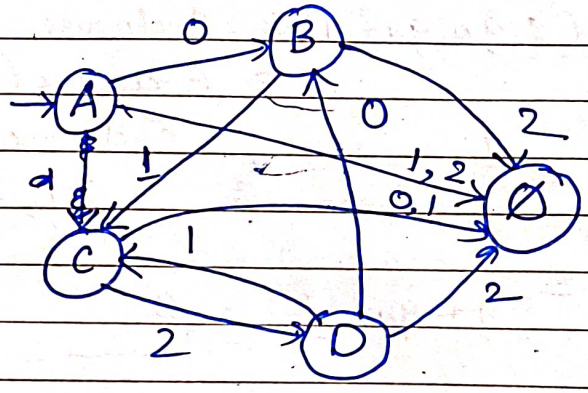
$\delta'(\{q_2\}, 2) = \epsilon$closure($\delta(\{q_2\}, 2)$)
$= \epsilon$closure(q_3)
$= \{q_3\}$ ——— New state

$\delta'(\{q_1, q_3, q_0\}, 0) = \epsilon\text{closure}(\delta(\epsilon\text{closure}(\{q_0, q_1, q_3\}, 0))$
$= \epsilon\text{closure}(\delta(\{q_0, q_1, q_3\}, 0))$
$= \epsilon\text{closure}(q_1)$
$= \{q_0, q_1\}.$

$\delta'(\{q_1, q_3, q_0\}, 1) = \epsilon\text{closure}(\delta(\epsilon\text{closure}(\{q_0, q_1, q_3\}, 1))$
$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_3\}, 1))$
$= \epsilon\text{-closure}(q_2)$
$= \{q_2\}$

$\delta'(\{q_0, q_1, q_3\}, 2) = \epsilon\text{-closure}(\delta(\epsilon\text{closure}(\{q_0, q_1, q_3\}, 2))$
$= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_3\}, 2))$
$= \epsilon\text{-closure}(\emptyset)$
$= \emptyset$

	0	1	2
A → $\{q_0\}$	$\{q_0, q_1\}$	$\emptyset$	$\emptyset$
B $\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_2\}$	$\emptyset$
C $\{q_2\}$	$\emptyset$	$\emptyset$	$\{q_0, q_1, q_3\}$
D $\{q_0, q_1, q_3\}$	$\{q_0, q_1\}$	$\{q_2\}$	$\emptyset$



MINIMIZATION OF DFA :-

→ Any DFA defines a unique language, but converse is not true.
→ For a single language, we may have multiple DFAs.
→ For a storage efficiency it is desirable to reduce the no. of states as far as possible.

Two states p, q of DFA are called indistinguishable if $\hat{\delta}(p,w) \in F \Rightarrow \hat{\delta}(q,w) \in F$ and $\hat{\delta}(p,w) \notin F \Rightarrow \hat{\delta}(q,w) \notin F$ where $w \in \Sigma^*$ and

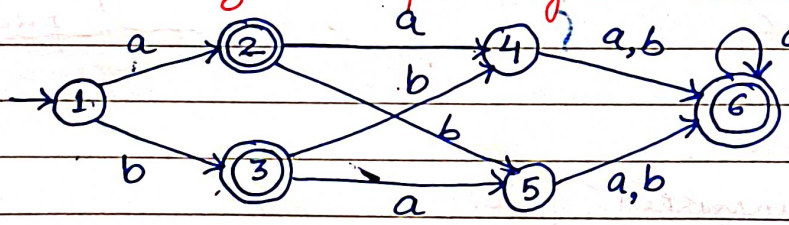
If $\hat{\delta}(p,w) \in F$ and $\hat{\delta}(q,w) \notin F$ or vice-versa then the states p & q are said to be distinguishable

MYHILL NERODE THEOREM :-

1. Remove all inaccessible states
2. Consides all pairs of states (p,q). If $p \in F$ & $q \notin F$ or vice-versa, mark the pair (p,q) as distinguishable.
3. Repeat the following step until no previously unmarked pairs are marked
   a) For all pairs (p,q) & all $a \in \Sigma$, compute $\delta(p,a) = pa$
      $\delta(q,a) = qa$
   b) If pair (pa, qa) is marked as distinguishable then (p,q) as distinguishable state.
4. Two or more states are equivalent if they are not marked.

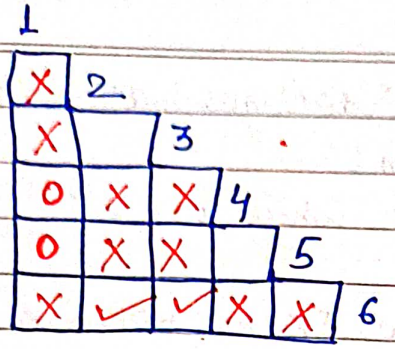
Example 1

Minimize the following automata



Sol.

First of all we will remove all inaccessible states since there is not state in given DFA which is inaccessible



Mark all state pairs which are distinguishable.

For eg.

1,2 — 1∉F , 2∈F
1,3 — 1∉F , 3∈F
2,4 — 2∈F, 4∉F
2,5 — 2∈F , 5∉F
3,4 — 3∈F . 4∉F
3,5 — 3∈F . 5∉F
1,6 — 1∉F, 6∈F
4,6 ..
5,6 ..
2,6 — both are final
3,6 — Finals
2,6 - Not marked
3,6 - Not marked
4,5 - Not marked

Here 4,6 is already marked in table
∴ 2,6 will be marked

Now For remaining unmarked pairs we will repeat step3 & check for input a & b

Iteration -1 (X)

δ(1,a) = 2
δ(1,b) = 3
δ(1,a) = 2
δ(1,b) = 3
δ(2,a) = 4
δ(2,b) = 5
δ(2,a) = 4
δ(3,a) = 5

δ(4,a) = 6
δ(4,b) = 6
δ(5,a) = 6
δ(5,b) = 6
δ(3,a) = 5
δ(3,b) = 4
δ(6,a) = 6
δ(6,a) = 6

5,6 is marked Hence 3,6 will also be marked

δ(4,a) = 6
δ(4,b) = 6

δ(5,a) = 6
δ(5,b) = 6

} 6,6 is not distinguishable pair

Iteration -2 (O)

Again check remaining unmarked pair.

δ(1,a) = 2
δ(1,a) = 2
δ(2,a) = 4
δ(2,b) = 5

δ(4,a) = 6
δ(5,a) = 6
δ(3,a) = 5
δ(3,b) = 4

Since 2,6 is marked.
again 2,6 is marked
4,5 is yet not marked

$\delta(4,a) = 6$ $\quad \delta(5,a) = 6$
$\delta(4,b) = 6$ $\quad \delta(5,b) = 6$
} 6,6 is not distinguishable pair

Iteration -2

Go for remaining pairs unmarked in table

$\delta(2,a) = 4$ $\quad \delta(3,a) = 5$
$\delta(2,b) = 5$ $\quad \delta(3,b) = 4$
} 4,5 is not marked.

$\delta(4,a) = 6$ $\quad \delta(5,a) = 6$
$\delta(4,b) = 6$ $\quad \delta(5,b) = 6$
} 6,6 is not distinguishable

As after Iteration -3, there is no change or state is marked, hence process will stop here.

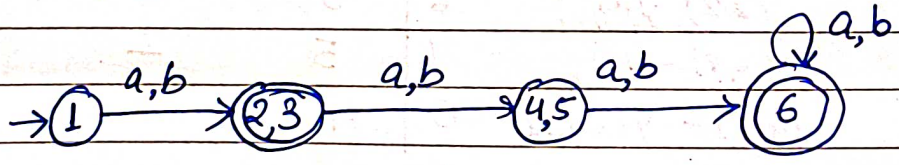
From the table, 2,3 is not marked

$\therefore 2 \approx 3$

4,5 is not marked

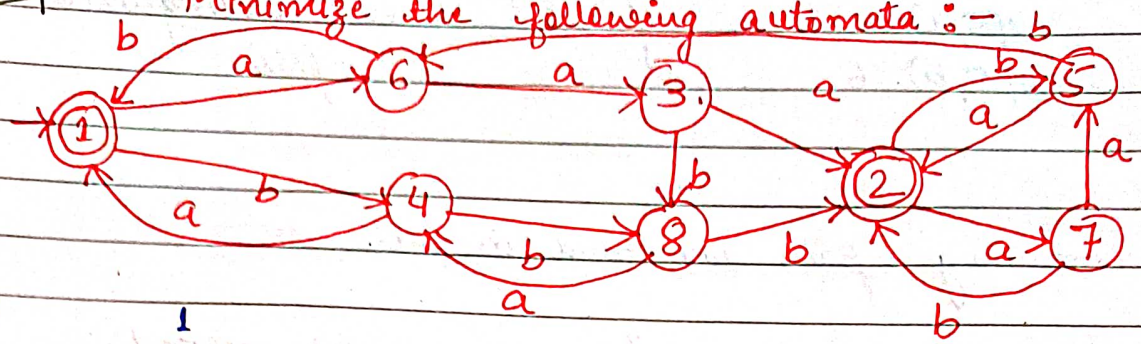
$\therefore 4 \approx 5$

Minimized DFA will be —



Example

Minimize the following automata :-



Sol^n

	2						
X	X	3					
X	X		4				
X	X			5			
X	X	✓	✓	✓	6		
X	X	✓	✓	✓		7	
X	X	✓	✓	✓			8
1							

First we mark all distinguishable pairs

Final states	Non-final states
1, 2	3, 4, 5, 6, 7, 8

(1,3), (1,4), (1,5), (1,6), (1,7), (1,8)
(2,3), (2,4), (2,5), (2,6), (2,7), (2,8)

Now we go for **Iteration -1**

$\delta(1,a) = 6$ $\delta(2,a) = 7$ (6,7) – Not marked
$\delta(1,b) = 4$ $\delta(2,b) = 5$ (4,5) – Not
$\delta(3,a) = 2$ $\delta(4,a) = 1$ (2,1) = No
$\delta(3,b) = 8$ $\delta(4,b) = 8$ (8,8) – Not Distinguish.
$\delta(3,a) = 2$ $\delta(5,a) = 2$ (2,2) – Not Distins.
$\delta(3,b) = 8$ $\delta(5,b) = 6$ (6,8) – Not mark
$\delta(3,a) = 2$ $\delta(6,a) = 3$ (2,3) – Yes Mark.
$\delta(3,a) = 2$ $\delta(7,a) = 5$ (2,5) = Yes marked
$\delta(3,a) = 2$ $\delta(8,a) = 4$ (2,4) – Yes mark.
$\delta(4,a) = 1$ $\delta(5,a) = 2$ (1,2) – Not
$\delta(4,b) = 8$ $\delta(5,b) = 6$ (6,8) – NO
$\delta(4,a) = 1$ $\delta(6,a) = 3$ (1,3) – marked
$\delta(4,a) = 1$ $\delta(7,a) = 5$ (1,5) – marked
$\delta(4,a) = 1$ $\delta(8,a) = 4$ (1,4) – marked

	a	b
→ (1)	6	4
(2)	7	5
3	2	8
4	1	8
5	2	6
6	3	1
7	5	2
8	4	2

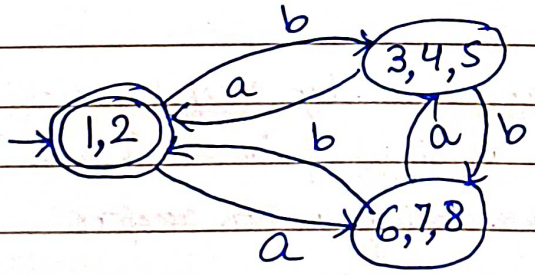
$\delta(5,a) = 2$	$\delta(6,a) = 3$	(2,3) - Yes
$\delta(5,a) = 2$	$\delta(7,a) = 5$	(2,5) - Yes
$\delta(5,a) = 2$	$\delta(8,a) = 4$	(2,4) - Yes
$\delta(6,a) = 3$	$\delta(7,a) = 5$	(3,5) - Not
$\delta(6,a) = 3$	$\delta(8,a) = 4$	(3,4) - Not

Iteration - 2

$\delta(1,a) = 6$	$\delta(2,a) = 7$	(6,7) - Not
$\delta(1,b) = 4$	$\delta(2,b) = 5$	(4,5) - Not
$\delta(3,a) = 2$	$\delta(4,a) = 1$	(1,2) - Not
$\delta(3,b) = 8$	$\delta(4,b) = 8$	(8,8) - Not distinguishable
$\delta(3,a) = 2$	$\delta(5,a) = 2$	(2,2) - Not distinguishable
$\delta(3,b) = 8$	$\delta(5,b) = 8$	(8,8) - Not distinguishable
$\delta(4,a) = 1$	$\delta(5,a) = 2$	(1,2) - Not
$\delta(4,b) = 8$	$\delta(5,b) = 8$	(8,8) - Not
$\delta(6,a) = 3$	$\delta(7,a) = 5$	(3,5) - Not
$\delta(6,b) = 1$	$\delta(7,b) = 2$	(1,2) - Not
$\delta(6,a) = 3$	$\delta(8,a) = 4$	(3,4) - Not
$\delta(6,b) = 1$	$\delta(8,b) = 2$	(1,2) - Not
$\delta(7,a) = 5$	$\delta(8,a) = 4$	(4,5) - Not
$\delta(7,b) = 2$	$\delta(8,b) = 2$	(2,2) - Not distinguishable

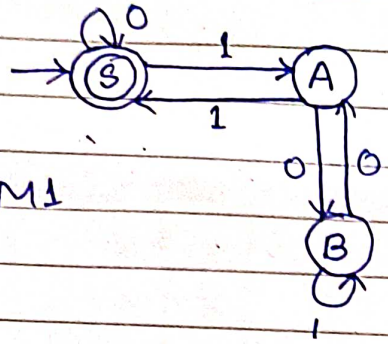
Since there is not changes marked in table :.

$1 \approx 2$, $3 \approx 4 \approx 5$, $6 \approx 7 \approx 8$.

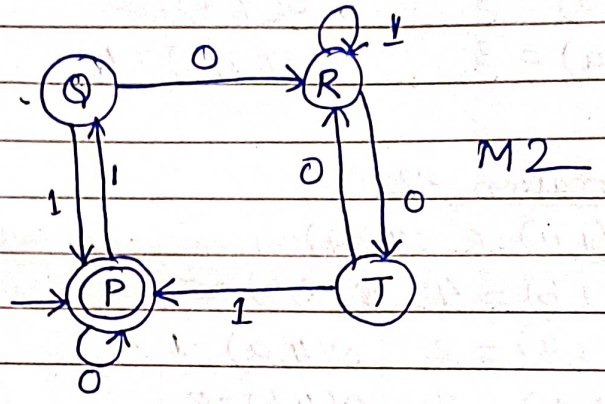


EQUIVALENCE OF DFA's :-

→ It is quite possible that two DFAs may be structurally different but their behaviour towards string acceptance may be the same.



M1



M2

→ M1 & M2 DFAs are structurally different but string acceptance for both DFAs are same.

→ To prove equivalence, draw comparison table with 0 & 1 as input symbols. Each state entry in this table is a pair of state one belonging to M1 & other belonging to M2.

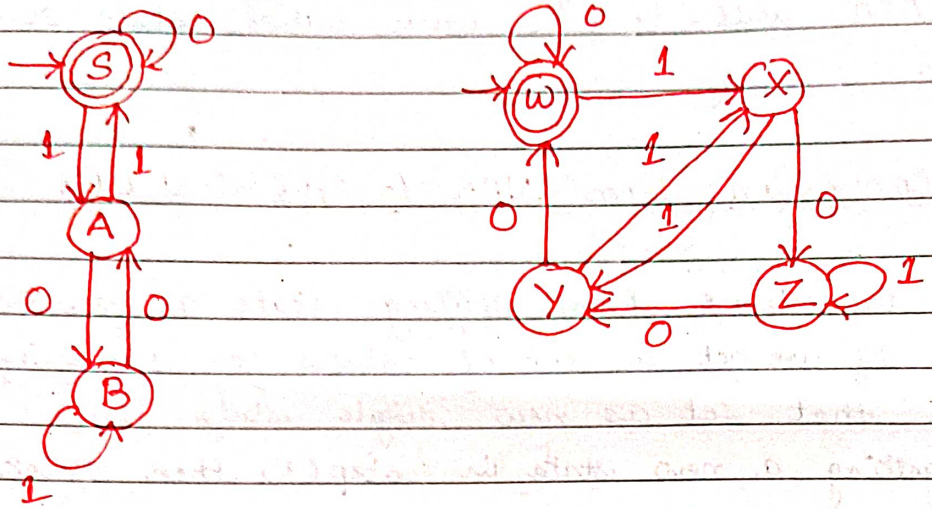
Comparison table for DFA M1 & M2

Current State	Input symbol 0	1
→ (S, P)	(S, P)	(A, Q)
(A, Q)	(B, R)	(S, P)
(B, R)	(A, T)	(B, R)
(A, T)	(B, R)	(S, P)

- The equivalence behaviour can be established by carefully looking at the comparison table; even if there is a single pair with one state being an acceptor state & the other being a non acceptor state, then two FA are not equivalent.

Example

check if two FAs are equivalent :-



Sol.^n

Comparison table for DFA 1 & DFA 2

Current state	0	1
(S, W)	(S, W)	(A, X)
(A, X)	(B, Z)	(S, Y)

Incompatible pair as S is final state
✗ is non final state

TRANSFORMATION OF NFA TO DFA :-

↳ For every NFA, there exist an equivalent DFA. It is defined in terms of language acceptance.

Since DFA equivalent of NFA is to simulate the moves in parallel state of DFA will be combination of one or more states of NFA, hence every state of DFA will be represented by some subset of set of states of NFA, hence every state of DFA will be represented by some subset of set of states of NFA, & therefore the transition of NFA to DFA is normally c/d a subset construction

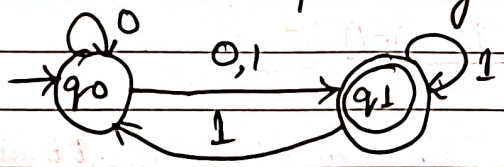
If a given NFA has n states, then no. of states of the equivalent DFA will be 2^n, with initial state corresponding to subset $\{q_0\}$.

Algorithm : Conversion from NFA to equivalent DFA

1. Seek all the transitions from starting state q_0 for every symbol in Σ. If we get a set of states for same input then consider that set as new single state.
2. If we are getting a new state in step(1) then check all transitions of Σ for this new state only. Suppose new state as $[q_1, q_2]$ then for some input alphabet 'a' compute $\delta(q_1, a) \cup \delta(q_2, a)$.
3. Repeat step 2 till we are getting any new state.
4. All those states which consist at least one final state of given NFA as member state will be considered as final state.

Example

Convert the following NFA into DFA :-



Sol.

Initially, we have q_0 as starting state so create start state as $[q_0]$

Now $\delta([q_0], 0) = \{q_0, q_1\}$ ——— New state

so we create new state as $\{q_0, q_1\}$.

Again we find $\delta([q_0], 1) = \{q_1\}$ ——— New state

Again it's a new state

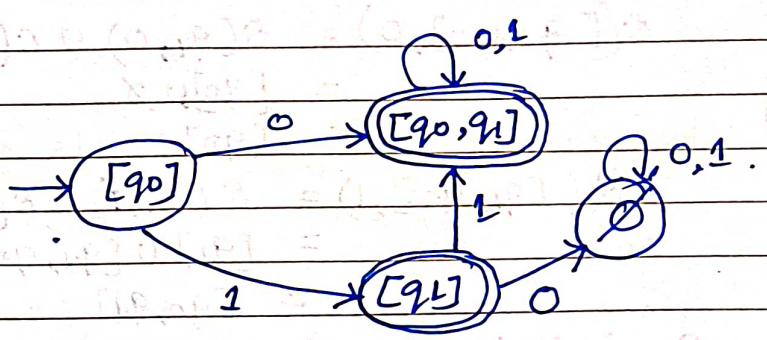
Next we find transitions for $\delta([q_0,q_1],0) = \delta(q_0,0) \cup \delta(q_1,0)$
$= [q_0,q_1] \cup \emptyset$
$= [q_0,q_1]$

$\delta([q_0,q_1],1) = \delta(q_0,1) \cup \delta(q_1,1)$
$= [q_0] \cup [q_0,q_1]$
$= [q_0,q_1]$

$\delta([q_1],0) = \emptyset$
$\delta([q_1],1) = [q_0,q_1]$

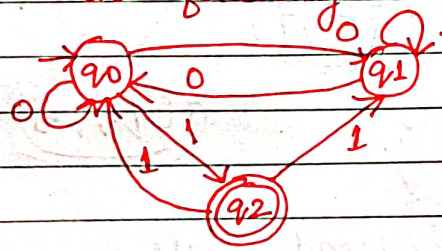
Equivalent DFA will be

	0	1
→ $[q_0]$	$[q_0,q_1]$	$[q_1]$
$[q_0,q_1]$	$[q_0,q_1]$	$[q_0,q_1]$
$[q_1]$	$\emptyset$	$[q_0,q_1]$
$\emptyset$	$\emptyset$	$\emptyset$



Finally, $[q_0,q_1]$ and $[q_1]$ contains final state q_1 of NFA so these are final states in equivalent DFA

Example Convert the following NFA to its equivalent DFA



Sol^n

Initial state is q_0

$\delta([q_0],0) = [q_0,q_1]$ – New
$\delta([q_0],1) = [q_2]$ – New

$\delta([q_0,q_1],0) = \delta(q_0,0) \cup \delta(q_1,0)$
$= [q_0,q_1] \cup [q_0]$
$= [q_0,q_1]$

$\delta([q_0,q_1],1) = \delta(q_0,1) \cup \delta(q_1,1)$
$= [q_2] \cup [q_1]$
$= [q_1,q_2]$ — New state

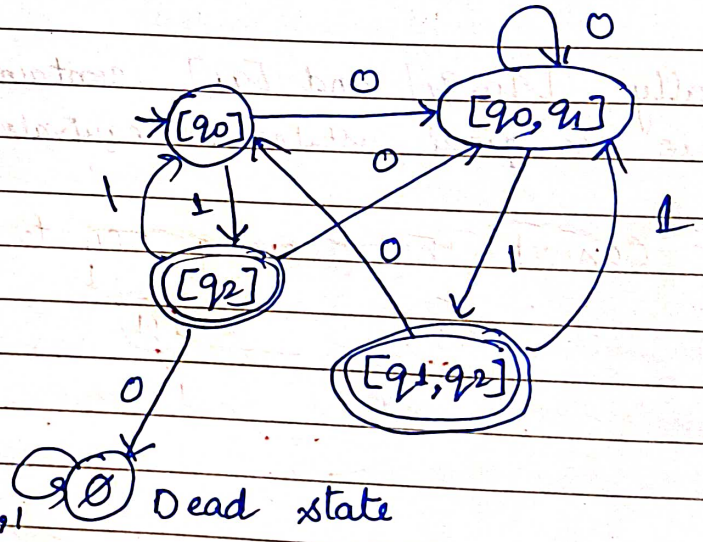
$\delta([q_2],0) = \emptyset$
$\delta([q_2],1) = [q_0,q_1]$

$\delta([q_1,q_2],0) = \delta(q_1,0) \cup \delta(q_2,0)$
$= [q_0] \cup \emptyset$
$= [q_0]$
$\delta([q_1,q_2],1) = \delta(q_1,1) \cup \delta(q_2,1)$
$= [q_1] \cup [q_1,q_0]$
$= [q_0,q_1]$

Finally, $[q_1,q_2]$ & $[q_2]$ has final state of q_2 of NFA
$\therefore [q_1,q_2]$ & $[q_2]$ are final states of equivalent DFA

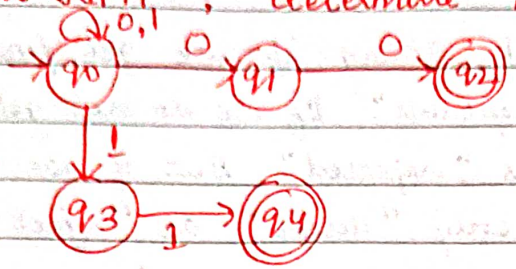
Equivalent DFA will be

	0	1
$\rightarrow [q_0]$	$[q_0,q_1]$	$[q_2]$
$[q_0,q_1]$	$[q_0,q_1]$	$[q_1,q_2]$
$*[q_2]$	$\emptyset$	$[q_0,q_1]$
$*[q_1,q_2]$	$[q_0]$	$[q_0,q_1]$
$\emptyset$	$\emptyset$	$\emptyset$



Example

Given NFA, determine the equivalent DFA



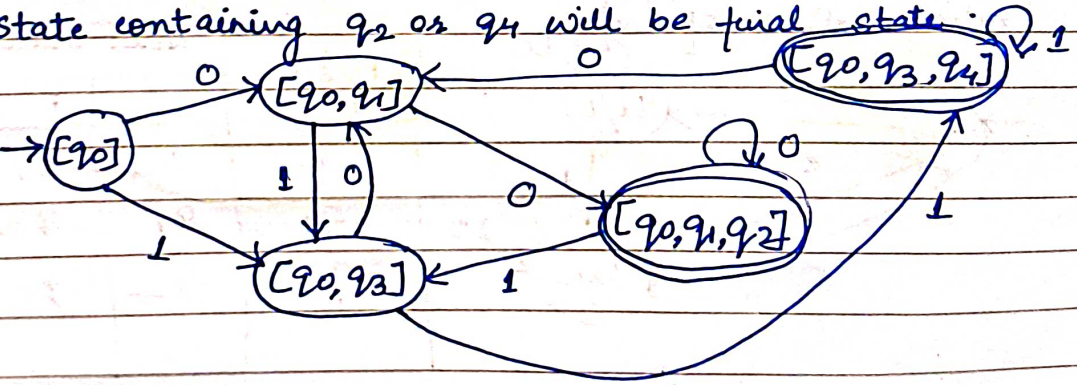
Sol.

State/Σ	0	1
→q0	{q0, q1}	{q0, q3}
q1	{q2}	∅
(q2)	∅	∅
q3	∅	{q4}
(q4)	∅	∅

Now, we construct state table of DFA through subset construction

State/Σ	0	1
→ {q0}	{q0, q1}	{q0, q3}
{q0, q1}	{q0, q1, q2}	{q0, q3}
{q0, q3}	{q0, q1}	{q0, q3, q4}
{q0, q1, q2}	{q0, q1, q2}	{q0, q3}
{q0, q3, q4}	{q0, q1}	{q0, q3, q4}

Any state containing q_2 or q_4 will be final state.



FINITE AUTOMATA WITH OUTPUT :-

In FA after reading the input string if we get final state then the string is said to be "acceptable". If we do not get final state then it is said that string is "rejected". But if we need to produce some precise information & not only "accept" or "reject" then we shall consider the finite set of output symbols & machine function as well.

There are 2 types of FA with output & those are:

1. Moore Machine
2. Mealey Machine.

1. Moore Machine

Moore m/c is a finite state m/c in which next state is decided by current state & current input symbol. The output symbol at a given time depends only upon the present state of m/c.

Formal Definition

Moore m/c is 6 tuple : $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where

Q : Finite set of states

Σ : Finite set of input alphabets

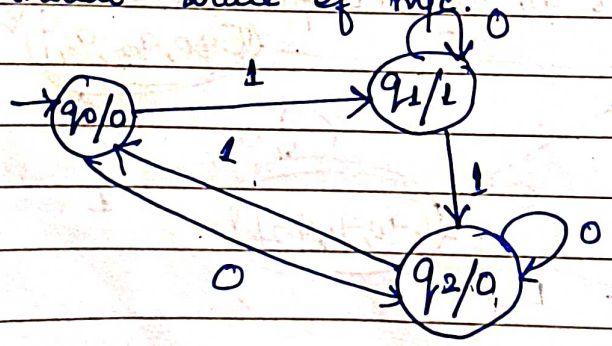
Δ : output alphabet

δ : Transition function $\Sigma \times Q \to Q$ Also known as state fun.^{c}.

λ : output function mapping $Q \to \Delta$

q_0 : initial state of m/c.

Ex:



Present State	Next State		Output
	0	1	
→q_0	q_2	q_1	0
q_1	q_1	q_2	1
q_2	q_2	q_0	0

2. Mealey Machine :-

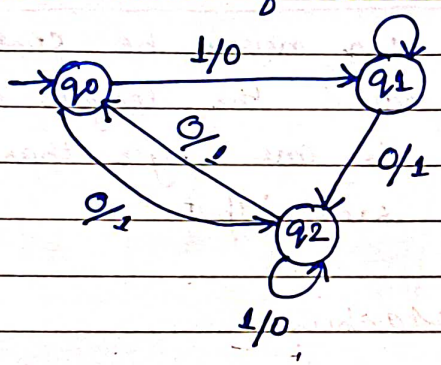
Mealey machine is a m/c in which output symbol depends upon the present input symbol & present state of machine.

Formal Definition

A Mealey m/c is a 6-tuple : $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$ where

- Q : Finite set of states
- Σ : Finite input symbols
- Δ : Finite output alphabet
- δ : $Q \times \Sigma \to Q$
- λ : Machine function defined as $\lambda : Q \times \Sigma \to \Delta$
- q_0 : Initial state of machine.

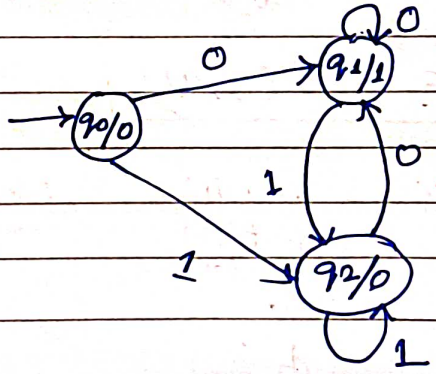
Ex,



Present State	Next state			
	0		1	
	state	o/p	state	o/p
→ q_0	q_2	1	q_1	0
q_1	q_2	1	q_1	0
q_2	q_0	1	q_2	0

Example Design a Moore machine to generate 1's complement of given binary number.

Sol.

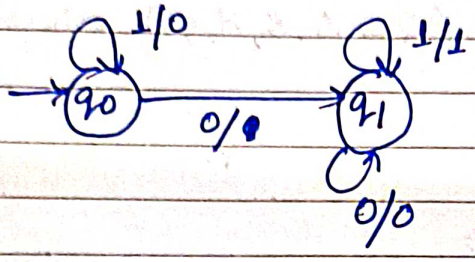


Example

Design a Mealey machine which will increment the given binary number by 1.

Sol:-

110011
110100 ← Read from this side bit by bit



100
+101

101
+ 1
110

EQUIVALENCE OF MOORE & MEALEY MACHINES

A Moore machine can never be correctly equivalent to Mealey machine because the length of output string from a Moore machine is one larger than that from a Mealey m/c by giving the same input.

Moore Machine to Mealey Machine

Let $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ be a Moore machine. The equivalent Mealey machine can be represented by $M' = (Q, \Sigma, \Delta, \delta, \lambda', q_0)$

output function $\boxed{\lambda'(q, a) = \lambda(\delta(q, a))}$

Ex, Convert the following Moore Machine to Mealey Machine

PS	0	1	output(λ)
→q0	q0	q1	0
q1	q2	q0	1
q2	q1	qe	2

Sol.ⁿ output function λ' can be obtained using following rule:

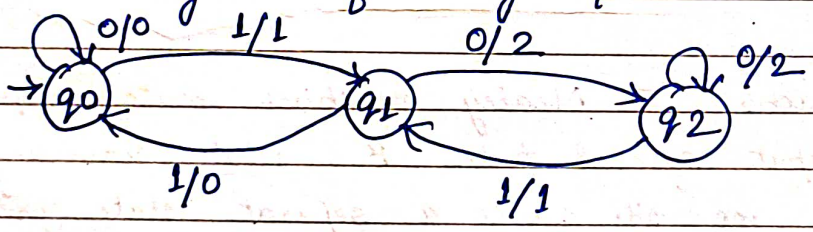
$$\boxed{\lambda'(q,a) = \lambda(\delta(q,a))}$$

$\lambda'(q_0,0) = \lambda(\delta(q_0,0)) = \lambda(q_0) = 0$
$\lambda'(q_0,1) = \lambda(\delta(q_0,1)) = \lambda(q_1) = 1$
$\lambda'(q_1,0) = \lambda(\delta(q_1,0)) = \lambda(q_2) = 2$
$\lambda'(q_1,1) = \lambda(\delta(q_1,1)) = \lambda(q_0) = 0$
$\lambda'(q_2,0) = \lambda(\delta(q_2,0)) = \lambda(q_1) = 1$
$\lambda'(q_2,1) = \lambda(\delta(q_2,1)) = \lambda(q_2) = 2$

Transition table can be drawn as

Present state	Next state			
	0		1	
	State	O/P	State	O/P
q_0	q_0	0	q_1	1
q_1	q_2	2	q_0	0
q_2	q_1	1	q_2	2

Transition diagram of Mealey M/c



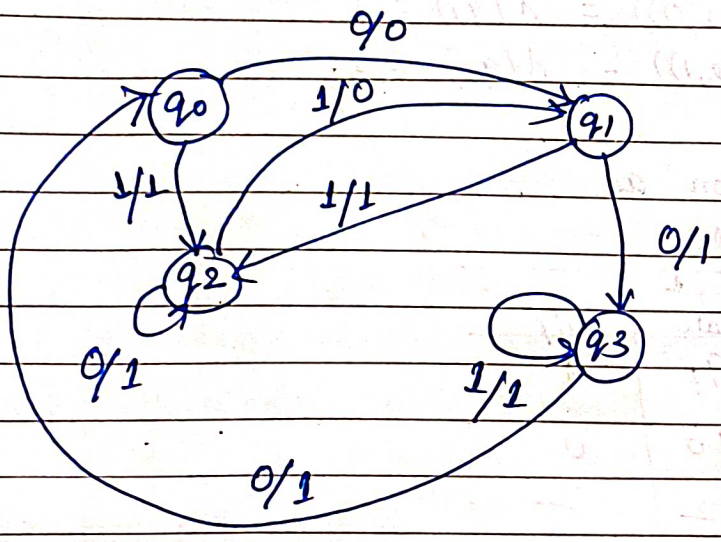
Example: Construct Mealey M/c equivalent to Moore M/c given by

Present state	Next state		output
	a=0	a=1	
→ q_0	q_1	q_2	1
q_1	q_3	q_2	0
q_2	q_2	q_1	1
q_3	q_0	q_3	1

Q6)

Transition table for Mealey machine can be drawn as

Present state	Next state			
	Input=0		Input=1	
	State	o/p	State	o/p
q_0	q_1	0	q_2	1
q_1	q_3	1	q_2	1
q_2	q_2	1	q_1	0
q_3	q_0	1	q_3	1



From Mealey Machine to Moore Machine :-

To convert Moore machine to Mealey machine state output symbols are distributed to input symbol paths. But while converting Mealey machine to Moore m/c we will create a separate state for every new output symbol & according incoming & outgoing edges are distributed.

If given Mealey m/c is $M_1 = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$ then an equivalent Moore machine will be

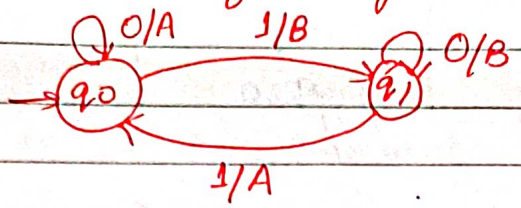
$$M_2 = ([Q \times \Delta], \Sigma, \Delta, \delta', \lambda', [q_0, b_0])$$

b_0 is arbitrarily selected member of Δ

$$\delta'([q_0, b], a) = [\delta(q, a), \lambda(q, a)]$$

$$\lambda'([q, b]) = b$$

Example Convert the following Mealey m/c into equivalent Moore M/c.



Sol.

Present state	Next state 0	Next state 1	Output
q_{0A}	q_{0A}	q_{1B}	A
q_{0B}	q_{0A}	q_{1B}	B
q_{1A}	q_{0B}	q_{1A}	A
q_{1B}	q_{0B}	q_{1A}	B

Example Consider the Mealey machine given by following table:

Present state	a=0 Next state	a=0 output	a=1 Next state	a=1 output
→q_1	q_3	1	q_2	0
q_2	q_1	1	q_4	1
q_3	q_2	0	q_1	1
q_4	q_4	1	q_3	0

Construct a Moore M/c

Sol. 4

Present state	Next state		Output
	a=0	a=1	
→ q_1	q_{31}	q_2	1
q_2	q_1	q_4	0
q_{31}	q_2	q_1	1
q_{30}	q_2	q_1	0
q_4	q_4	q_{30}	1

Applications of Automata Theory :-

1. Automata theory plays an important role in compiler design.
2. Automata theory is the base for the formal languages & these formal languages are useful for programming languages.
3. To prove the correctness of the program automata theory is used.
4. In design & analysis of digital circuits automata theory is used.